ALPHA MICROSYSTEMS
RIGHT. FROM THE START.

AMOS®
Assembly Language Programmer's Reference Manual

DSM-00052-02

© 1996 Alpha Microsystems

REVISIONS INCORPORATED	
REVISION	DATE
00	March 1988
01	April 1991
02	September 1996

AMOS Assembly Language Programmer's Reference Manual
To re-order this document, request part number DSO-00052-00.

This document applies to AMOS version 2.3 and later.

The information contained in this manual is believed to be accurate and reliable. However, no responsibility for the accuracy, completeness or use of this information is assumed by Alpha Microsystems.

This document may contain references to products covered under U.S. Patent Number 4,530,048.

The following are registered trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AMIGOS	AMOS	Alpha Micro	AlphaACCOUNTING
AlphaBASIC	AlphaCALC	AlphaCOBOL	AlphaDDE
AlphaFORTRAN 77	AlphaLAN	AlphaLEDGER	AlphaMAIL
AlphaMATE	AlphaNET	AlphaPASCAL	AlphaRJE
AlphaWRITE	CASELODE	OmniBASIC	VER-A-TEL
VIDEOTRAX			

The following are trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AlphaBASIC PLUS	AlphaVUE	AM-PC	AMTEC
AlphaDDE	AlphaConnect	DART	*in*Sight/am
*in*Front/am	ESP	MULTI	

All other copyrights and trademarks are the property of their respective holders.

ALPHA MICROSYSTEMS
2722 S. Fairview St.
P.O. Box 25059
Santa Ana, CA 92799

Table of Contents

CHAPTER 1 - INTRODUCTION

1.1 COMPATIBILITY NOTE 1-2
1.2 CONTENTS OF THIS BOOK 1-2
1.3 READER'S COMMENTS FORM 1-3
1.4 RELATED BOOKS 1-3
1.5 GRAPHIC CONVENTIONS USED IN THIS BOOK 1-4

Part 1
Introduction to Alpha Micro Assembly Language Programming

CHAPTER 2 - FILES USED IN THE ASSEMBLY LANGUAGE SYSTEM

2.1 .M68 - SOURCE FILE 2-1
2.2 .OBJ - INTERMEDIATE OBJECT FILE 2-1
2.3 .LIT - BINARY PROGRAM (LITERAL) FILE 2-2
2.4 .OVR - BINARY OVERLAY FILE 2-2
2.5 .LST - PROGRAM LISTING FILE 2-2
2.6 .LIB - LIBRARY FILE 2-3
2.7 .GLB - GLOBAL CROSS REFERENCE FILE 2-3
2.8 .MAP - LOAD MAP FILE 2-3
2.9 .SYM - RESOLVED SYMBOL FILE 2-3
2.10 .IPF - INTERPHASE WORK FILE 2-3
2.11 .TMP - TEMPORARY WORK FILE 2-4
2.12 .UNV - UNIVERSAL FILE 2-4
2.13 .RTI - RUN TIME INITIALIZER FILE 2-4

CHAPTER 3 - M68 SOURCE PROGRAM FORMAT

3.1 MACHINE INSTRUCTIONS 3-2
3.2 DATA GENERATION STATEMENTS 3-3
3.3 SYMBOLIC EQUATE STATEMENTS 3-3
3.4 ASSEMBLY CONTROL STATEMENTS 3-4
3.5 CONDITIONAL ASSEMBLY DIRECTIVES 3-4
3.6 MACRO DEFINITIONS AND MACRO CALLS 3-5
3.7 COMMENT LINES AND BLANK LINES 3-5

CHAPTER 4 - TERMS AND EXPRESSIONS

4.1 CHARACTER SET 4-1
4.2 TERMS 4-2
4.3 EXPRESSIONS 4-3
4.4 NUMBERS 4-5
4.5 REGISTER SYMBOLS 4-6
4.6 ASSEMBLY LOCATION COUNTER 4-6
4.6.1 Assembly Sections 4-7
4.7 LOCAL SYMBOLS 4-11

CHAPTER 5 - ASSEMBLER PSEUDO OPCODES

5.1 ASSEMBLY CONTROL PSEUDO OPCODES 5-1
5.1.1 COPY 5-1
5.1.2 SEARCH 5-3
5.1.3 OBJNAM 5-4
5.1.4 PAGE 5-5
5.1.5 LIST and NOLIST 5-5
5.1.6 ASECT, PSECT and DSECT 5-6
5.1.7 DSIZE 5-6
5.1.8 SYM and NOSYM 5-6
5.1.9 CREF, NOCREF and MAYCREF 5-7
5.1.10 EVEN 5-7
5.1.11 RADIX 5-7
5.1.12 NVALU 5-7
5.1.13 END 5-8
5.1.14 ASMMSG 5-8
5.1.15 ASMERP 5-9
5.2 DATA GENERATION PSEUDO OPCODES 5-9
5.2.1 BYTE 5-9
5.2.2 WORD 5-10
5.2.3 WORD32 5-10
5.2.4 WORD64 5-10
5.2.5 LWORD 5-10
5.2.6 ASCII 5-11
5.2.7 ASCIZ 5-11
5.2.8 RAD50 5-11
5.2.9 OFFSET 5-11
5.2.10 BLKB, BLKW, BLKL 5-12
5.3 SEGMENTATION PSEUDO OPCODES 5-12
5.3.1 Segmenting Assembly Language Programs 5-12
5.3.2 AUTOEXTERN 5-14
5.3.3 INTERN 5-14
5.3.4 EXTERN 5-15
5.3.5 UNIVERSAL 5-15
5.3.6 OVRLAY 5-16

CHAPTER 6 - USER DEFINED MACROS

6.1 MACRO DEFINITION 6-1
6.1.1 Macro Definition Formats 6-2
6.1.2 The Macro Source Statements 6-2
6.1.3 The Dummy Argument List 6-3
6.1.4 Labels 6-3
6.1.5 Local Symbols 6-3
6.1.6 Comments 6-4
6.1.7 Special Macro Operators 6-4
6.1.8 Suppressing Macro Expansion - ENDMX 6-6
6.1.9 NCHR, NTYPE, NEVAL and NSIZE 6-6
6.1.10 Sample Macro Definitions 6-8
6.2 MACRO CALLS 6-8
6.2.1 Name 6-9
6.2.2 Real Arguments 6-9
6.2.3 Label 6-10
6.2.4 Comments 6-11
6.2.5 Nested Macro Calls 6-11
6.2.6 Sample Macro Calls 6-11

CHAPTER 7 - CONDITIONAL ASSEMBLY DIRECTIVES

7.1 CONDITIONAL DIRECTIVE FORMATS 7-1
7.2 CONDITION CODES 7-2
7.3 SUBCONDITIONALS 7-3
7.4 NESTING OF CONDITIONALS 7-3

Part 2
Using the Alpha Micro Assembly Language Programming System

CHAPTER 8 - THE ALPHA MICRO ASSEMBLER (M68)

8.1 THE M68 PHASES 8-1
8.2 COMMAND LINE 8-2
8.2.1 Assembler Options 8-2
8.3 USING M68 8-4
8.3.1 Parameterized Assembly Option 8-5
8.4 SAMPLE ASSEMBLY DISPLAY 8-6
8.5 THE ASSEMBLY LISTING 8-6
8.5.1 Assembly Listing Format 8-7
8.5.2 Listing Control Pseudo Opcodes 8-7
8.5.3 Generating a Cross Reference 8-8
8.6 M68 ERRORS 8-10
8.6.1 Error Codes 8-10
8.6.2 Error Messages 8-11

CHAPTER 9 - LINKAGE EDITOR (LNKLIT) AND SYMBOL TABLE FILE GENERATOR (SYMLIT)
9.1 LNKLIT 9-1
9.1.1 LNKLIT Command Line 9-2
9.1.2 Sample LNKLIT Display 9-4
9.1.3 LNKLIT Errors 9-4
9.2 THE SYMBOL TABLE FILE GENERATOR (SYMLIT) 9-4
9.2.1 SYMLIT Command Line 9-5
9.2.2 Sample SYMLIT Display 9-7
9.3 LIBRARY AND OPTIONAL FILES 9-7
9.3.1 Library Files 9-8
9.3.2 Optional Files 9-8
9.4 THE LOAD MAP FILE 9-8
9.5 THE RUN TIME INITIALIZER FILE 9-9
9.6 LNKLIT AND SYMLIT ERROR MESSAGES 9-10

CHAPTER 10 - THE OBJECT FILE LIBRARY GENERATOR (LIBLIT)
10.1 LIBLIT COMMAND LINE 10-1
10.1.1 Continuation Lines 10-2
10.1.2 LIBLIT Option Switch (/L) 10-3
10.2 SAMPLE LIBLIT DISPLAY 10-3
10.3 UPDATING A LIBRARY 10-4
10.4 LIBLIT ERROR MESSAGES 10-4

CHAPTER 11 - THE GLOBAL CROSS REFERENCE GENERATOR (GLOBAL)
11.1 GLOBAL COMMAND LINE 11-1
11.1.1 Continuation Lines 11-2
11.1.2 GLOBAL Options 11-2
11.2 SAMPLE GLOBAL DISPLAY 11-2
11.3 SAMPLE LISTING DISPLAY 11-3
11.4 GLOBAL ERROR MESSAGES 11-4

APPENDIX A - SUMMARY OF PROGRAM OPTIONS
A.1 M68 . . . . . A-1
A.2 LNKLIT A-3
A.3 SYMLIT A-4
A.4 LIBLIT A-5
A.5 GLOBAL A-5

APPENDIX B - ASCII CHART

APPENDIX C - RAD50 CONVERSION
C.1 USING THE CONVERSION CHART C-2
C.2 CONVERSION CHART FOR PACKING AND UNPACKING RAD50 DATA C-5

APPENDIX D - SPECIAL ISSUES ON THE MC68040
D.1 THE PROBLEM AND WHY IT'S A PROBLEM D-1
D.2 WHAT YOU MUST DO D-2
D.3 ONE MORE CAUTION D-2

APPENDIX E - USING AMSORT.SYS
E.1 LOCATING AND VERIFYING AMSORT.SYS E-2
E.2 THE IMPURE AREA E-2
E.3 SETTING THE RECORD SIZE E-3
E.4 SETTING FLAGS E-3
E.5 INPUT AND OUTPUT ROUTINES E-4
E.6 EXAMPLE PROGRAM E-4

DOCUMENT HISTORY

INDEX

CHAPTER 1

INTRODUCTION

Alpha Micro computer systems support a flexible and efficient assembly language development system under the AMOS monitor. This system includes the assembler, linkage editor, symbol file generator, object file library generator, global symbol cross reference generator, and symbolic debugger.

The assembler, M68, is a multi-pass macro assembler with conditional assembly directives, library copy function, and external segment links. The linkage editor, LNKLIT, links multi-segment programs together and creates a runnable program file. The operating system supports segment overlays, allowing a large program to be traced and debugged as symbolic instructions using all the labels as they were entered in the source program. The library generator, LIBLIT, provides a mechanism for developing and maintaining a library file that contains frequently used routines, making them accessible to all programmers on the computer. GLOBAL provides a global cross reference listing for a group of object files. All components of the assembly language development system run under control of the standard AMOS monitor.

Over a hundred monitor calls currently exist in macro form that the assembly language programmer can use to communicate with the AMOS monitor and to use the routines it has to offer. These macro calls are predefined in a file named SYS.M68 located in account [7,7] on the AMOS System Disk. To include this entire library in your assembly language program, use a single COPY statement—then refer to the monitor call you want to use by its macro name. The monitor call system makes for an easy-to-use link to operating system resources. SYS.M68 also includes equate statements for many of the predefined system variables, including the job table entries for the user impure job variables.

For your programs to be compatible with AMOS architecture, you must write them in totally relocatable code. A relocatable program may be loaded anywhere in memory and executed without modifying any addresses within the program itself. (Relocatable programs are also called ***position independent***.) There are machine instructions which assist in writing totally relocatable code, and by obeying a few simple restrictions, writing assembly language programs for the Alpha Micro computer soon becomes second nature.

Optionally, you may write re-entrant programs, and then incorporate these programs or subroutines into system memory to be shared by all users without requiring a separate copy in each user's area of memory. (To add programs to system memory, you modify the system initialization command file. For information on the system initialization command file, see the *System Operator's Guide*.)

We will not go into the rules for re-entrant programming in great detail here since this is an advanced programming technique and requires specific rules that are not machine dependent. There are, however, a number of features in the instruction set that do lend themselves quite nicely to writing re-entrant code. Many books on the subject are available—all the general practices discussed in such books apply to the programming of the Alpha Micro computer system.

1.1 COMPATIBILITY NOTE

As of AMOS Version 2.0, significant changes and improvements were made to the entire assembly language system. These programs are "downward compatible"—that is, they will accept files generated by pre-AMOS 2.0 versions of the assembly language system. However, you cannot go the other direction—pre-AMOS 2.0 versions of the assembly language system will not accept files generated by later versions of the assembly language system.

In general, all releases of the assembly language system are downward, but not upward, compatible. For this reason, do not "mix" different versions of the assembly language system (e.g., a SYSLIB.LIB file from one AMOS release with a LNKLIT object file linkage editor from another).

68020, 68030, 68881 Extensions

Be cautious when incorporating addressing modes and features specific to 68020, 68030, and other microprocessors newer than the 68000 into your .M68 source files, since doing so will make your program files incompatible with earlier microprocessors. For information on the instructions and addressing modes available for 68020, 68030, 68881, and other processors used on the Alpha Micro computer, see the latest version of the *Alpha Micro Instruction Set Manual*.

See Chapter 8 for information on the M68 /M20 switch that is required when assembling a program containing 68020-specific instructions and addressing modes.

1.2 CONTENTS OF THIS BOOK

Part 1: Introduction to Alpha Micro Assembly Language Programming

Chapters 2 through 7 contain information on the form of your assembly language programs. Chapter 2 discusses the different types of files used by the Alpha Micro assembly language system. Chapter 3 discusses the format of your assembly language source program. Chapter 4 discusses labels, terms, and expressions in your assembly language program statements. Chapter 5 discusses the pseudo opcodes available to you. Chapter 6 discusses how to construct and call macros. Chapter 7 discusses how to use the conditional assembly feature of the assembler.

Part 2: Using the Alpha Micro Assembly Language Programming System

Chapters 8 through 11 give operating information for the various components of the Alpha Micro assembly language programming system:

PROGRAM	FUNCTION	CHAPTER
M68	The macro-assembler	Chapter 8
LNKLIT	The linkage editor	Chapter 9
SYMLIT	The symbol table generator	Chapter 9
LIBLIT	The object file library generator	Chapter 10
GLOBAL	The global cross reference generator	Chapter 11

Appendix A gives a brief summary of all option request switches used by M68, LNKLIT, SYMLIT, LIBLIT, and GLOBAL. Appendix B gives the complete ASCII character set, with values in decimal, octal, and hexadecimal. Appendix C gives a RAD50 conversion chart. (RAD50 is a special packed data format AMOS uses internally to encode many important pieces of system data.)

1.3 READER'S COMMENTS FORM

Please note the Reader's Comments Form at the back of this book. We depend on you for information on how to organize and clarify our documentation to be of most use to you. Please take a moment to fill out and mail this form with your comments. Any suggestions for future documentation projects are welcome.

1.4 RELATED BOOKS

For information on other components of the Alpha Micro assembly language programming system not covered in this book, refer to:

AMOS Monitor Calls Manual
AlphaFIX User's Manual
Alpha Micro Instruction Set Manual

(AlphaFIX is a screen-oriented debugger for Alpha Micro assembly language programs.)

1.5 GRAPHIC CONVENTIONS USED IN THIS BOOK

As with other books in the Alpha Micro documentation library, this book uses certain standard conventions to make our examples easier to use and understand.

Graphics Conventions

SYMBOL	MEANS
Number base	Unless otherwise stated, all number bases are decimal (base 10).
Account number	A disk account number consists of a project number followed by a programmer number, and identifies a specific disk account. For example, [200,2] is an account number. Also represented by the symbol [p,pn].
Filespec	A file specification. It usually has the elements: `Devn:Filename.Ext[p,pn]` where ***Devn:*** is a device specification identifying a logical unit of a physical device (such as a hard disk), ***filename*** gives the name of the file, ***ext*** specifies the file's extension, and ***[p,pn]*** gives the account number of the file.
`{°}`	Braces indicate an optional part of a command line. When these symbols appear in a sample command line, they designate elements you may omit.
TEXT	In an example of user/computer communication, bold text like this indicates the characters you type.
`text`	In an example of user/computer communication, text like this indicates the characters the computer displays on the terminal screen.
KEY	In an example, this symbol shows when to press the indicated key on your terminal keyboard. For example, the RETURN symbol tells you where in an example you must press the RETURN key. An ESCAPE symbol indicates where in an example to press the ESCAPE key (sometimes labeled ALT MODE or ESC).
^	This symbol indicates a control-character. For example, if you type a Control-C, you see it echoed on your terminal as `^C`.

Part 1
Introduction to Alpha Micro Assembly Language Programming

Chapters 2 through 7 introduce the experienced assembly language programmer to assembly language programming for the Alpha Micro 680XX-series computer.

CHAPTER 2

FILES USED IN THE ASSEMBLY LANGUAGE SYSTEM

This section describes the files used during the normal course of building and testing an assembly language program. We refer to these files by their extensions. For example, an .M68 file is any file with an extension of "M68." All files described here are not necessarily used by all programmers during any one programming session, but you will eventually run across all of them at one time or another so you might as well know what they are used for and how they are created.

Be careful what extensions you assign to your files. As you read the following paragraphs, you will notice that various processes erase files with certain extensions from the disk. For example, if you have a file named MINE.LIT and you assemble another file named MINE.M68, a new file MINE.LIT is created, causing the original MINE.LIT file to disappear. For this reason, you should only use the extensions below for the types of files for which they were intended. As another example, it is not a good idea to use the .OBJ extension for a file containing source text—assembling a .M68 file with the same name causes your text file to disappear.

2.1∞.M68 - SOURCE FILE

.M68 files are the original ASCII source files you create using the AlphaVUE editor. .M68 files are input files for the assembler program (M68) which makes one or more passes over them depending on the assembly options selected. If you want to make any changes to a program, you make the changes to the .M68 file using AlphaVUE—you then reassemble and relink it. Files you include with the COPY assembly pseudo opcode must also be ASCII source files, usually having an extension of .M68.

2.2∞.OBJ - INTERMEDIATE OBJECT FILE

.OBJ files are the direct output of the assembler (Phase 2) and contain the assembled binary code, symbol references, internal symbol definitions, and unresolved external symbol references. .OBJ files are not directly usable for anything by themselves—they must first be processed by one or more programs to get a finished file that can be used directly. The linkage editor program (LNKLIT) reads one or more .OBJ files and creates a fully resolved and runnable binary program file in memory image format. The library

generating program (LIBLIT) combines specified .OBJ files into an object file library. The GLOBAL program reads .OBJ files and creates a global symbol cross reference file. The symbol file program (SYMLIT) reads the .OBJ files and creates a file which contains all user defined symbols and their resolved addresses. This symbol table file is used by the symbolic debugger program AlphaFIX. The assembler itself re-reads the .OBJ file during Phase 3 together with the .M68 source file to create the ASCII listing file.

2.3∞.LIT - BINARY PROGRAM (LITERAL) FILE

.LIT files are created by the linkage editor program (LNKLIT) and are the end result of the assembly process. The .LIT file is a binary memory image of the assembled program which is loaded into user memory when the program is requested for execution. The .LIT file is the final, fully assembled and resolved machine language program of which the .M68 file was the source.

The .M68 file from which the .LIT file was generated must have been written using the rules for totally relocatable code so the .LIT file may be loaded into any memory location and executed without modification. One or more .OBJ files may have been input to the linker for the creation of the single .LIT file. Once you have tested the .LIT program file, you may place it into the System Program Library account, DSK0:[1,4], where it will become available to all users on the system.

2.4∞.OVR - BINARY OVERLAY FILE

If the program contains overlay segments which do not all reside in memory at the same time, the linkage editor generates one .LIT main segment file and one or more .OVR overlay segment files. LNKLIT generates each overlay file in response to an OVRLAY assembler pseudo opcode. The .LIT program segment will be responsible for calling and executing each of the .OVR segments during the running of the program. Your program may selectively bypass overlay segments as does the assembler itself, which contains six overlays. Overlay files have the same memory image format as the .LIT program files except they are resolved at an effective address other than zero so they will not completely overlay the controlling segment.

This addressing is the direct responsibility of the programmer—for more information on creating overlays, see Chapter 5.

2.5∞.LST - PROGRAM LISTING FILE

An optional output of the assembler is a complete, resolved listing of the source program with the associated binary code that was generated. M68 creates this listing file during Phase 3 of the assembly process—you may generate it directly from the .M68 and .OBJ files by bypassing Phases 1 and 2 with the /O assembly switch. The .LST file is formatted ASCII—you may display it via the TYPE command or examine it with AlphaVUE. You may also print the listing file using the PRINT or PRNT command.

2.6∞.LIB - LIBRARY FILE

The .LIB file is a library file. A library file contains a collection of .OBJ files that are linked into the main program as required. The LIBLIT program allows you to generate and maintain object file libraries. The LNKLIT and SYMLIT programs accept these library (.LIB) files as input and automatically include any object files from such a library necessary to resolve external references. See Chapter 10 for information on creating and maintaining program libraries.

2.7∞.GLB - GLOBAL CROSS REFERENCE FILE

The GLOBAL program reads a group of .OBJ files and creates an alphabetic cross reference .GLB file listing all global symbols in the files, and shows which files define them and which files accept them as externally defined symbols. For information on GLOBAL, see Chapter 11.

2.8∞.MAP - LOAD MAP FILE

Both the linkage editor LNKLIT and the symbol table file generator SYMLIT generate a load map file in response to the optional /M switch. The load map (.MAP) file shows how the assembled and linked object files will be located in memory when the program is loaded into memory prior to execution. It also gives information about each object file linked into the final .LIT file. For information on the load map, see Chapter 10.

2.9∞.SYM - RESOLVED SYMBOL FILE

The .SYM file is a direct output of the symbol file generation program (SYMLIT) which takes one or more object (.OBJ) files and creates a symbol table with all user defined symbols and their resolved machine addresses. The .SYM file is used as input to the debugger program AlphaFIX which may then operate with references to the user symbols in the program instead of absolute machine addresses. On a computer where the program is always offset by some amount in memory, this is almost essential if you are to be able to trace the execution flow of a program under test. The .SYM file is useful only for input to AlphaFIX. The LNKLIT program can also generate a .SYM symbol table file.

2.10∞.IPF - INTERPHASE WORK FILE

The .IPF file is a temporary work file built during the assembly process by Phase 1 of the assembler to carry information on to Phase 2. The .IPF file is packed binary and the only reason we mention it here is, on rare occasions (for example, the computer is accidentally rebooted during the middle of an assembly), you may find a .IPF file left on your disk. Erase it—it is useless and just takes up space. There is no problem if it exists and you don't find it, since the next assembly of the same program will erase any .IPF file it finds during Phase 1 before attempting to create a new one.

2.11∞.TMP - TEMPORARY WORK FILE

The LIBLIT program creates a temporary work file named Jobnam.TMP (where "Jobnam" is the name of your job). As with the .IPF file, you should never see this file unless something goes wrong. The next time you run LIBLIT, the .TMP file should disappear.

2.12∞.UNV - UNIVERSAL FILE

M68 generates a .UNV file when your source file contains a UNIVERSAL statement. This source file must contain only macro and symbol definitions, and must generate no code. By using the SEARCH statement, a source file can refer to a specific .UNV file, allowing the macro and symbol definitions from that .UNV file to be included in the source file assembly at the point where the SEARCH statement occurs. See Chapter 5 for more information on the UNIVERSAL and SEARCH statements.

2.13∞.RTI - RUN TIME INITIALIZER FILE

LNKLIT and SYMLIT create an .RTI file when you specify the /D switch. The .RTI file is processed when your program executes the GETRTI monitor call. See the manual *AMOS Monitor Calls* for details on how to use the GETRTI monitor call.

CHAPTER 3

M68 SOURCE PROGRAM FORMAT

An M68 source program is a single .M68 file composed of a sequence of ASCII character source statement lines. Each line must be complete in itself since there is no provision for multiple-line statements. Each statement may be one of the following, depending on its function:

- Valid machine instruction
- Data generation statement
- Symbolic equate statement
- Assembly control statement
- Conditional assembly directive
- Macro definition
- Macro call
- Comment or blank line

The maximum line length is 100 characters. Each line is terminated by a carriage-return and line-feed which the text editor provides when you press RETURN. Unless otherwise specified, all of the lines listed above may contain an optional comment field following the actual statement—this comment field starts with a semicolon (;) and extends to the end of the line. Comments are ignored by the assembler—their purpose is to document the source code of your program so you can reconstruct at a later time exactly what you had in mind when you wrote the program. As such, they are extremely important, and you should use them liberally.

The assembler treats spaces and tabs (Control-I) as equal—they are used to delimit fields within statements. Tabs are useful to keep statement fields aligned and make for clean listings. Tabs are an important part of generating readable code.

The term "user symbol" occurs several times throughout this book, so we will define it at this point. A user symbol is any name defined by you within your program. It must be defined only once in that program, and can be from one to 32 characters in length.

Legal characters for a user symbol include the letters A-Z and a-z, the numbers 0-9, and the special characters ".", "%," and "$". Lower case letters will be "folded" to upper case if you don't use the assembler /N switch.

3.1 MACHINE INSTRUCTIONS

One machine statement is allowed per line and is assembled into a single machine hardware instruction which generates one, two, or three words of binary code depending on the instruction and addressing modes used. The general format of a machine instruction statement is:

```
{label:}   {opcode}   {operands} {;comments}
```

The label field is optional and is used to give a symbolic name to the location of the current instruction being assembled. It must terminate with a colon. The label may be any valid user symbol not previously defined. The value of the label may be either absolute or relocatable depending on the current assembly status. Relocatable symbols will be resolved during link-edit time by adding the label value to the current program relocation bias (calculated by LNKLIT). More than one label may appear on the same statement line separated by colons—in this case, each label is given the same value as the current location. Any symbol used in a label field may not be redefined later in the program. A label may appear as the only item on a line in which case it is assigned the address of the next byte of generated code.

The opcode field is required and contains one of the machine instruction opcodes in mnemonic form such as MOV, CLRB, TST, ADD, etc. (Refer to the *Instruction Set Manual* for a complete description of all the machine instructions available on the Alpha Micro computer.) The opcode field terminates with a space, tab, semicolon or carriage-return. If a label field was used, a space or tab between the colon and the opcode is optional but recommended.

The operands field is required on those instructions that have either one or two operands. The operands field is separated from the opcode field by one or more spaces or tabs. If the instruction being used requires two operands, the operands are separated from each other by a comma. Leading spaces are always ignored in the operands field while the operands themselves terminate with a space, tab, comma, semicolon or carriage-return.

The comments field is optional and is defined by a leading semicolon. The comments field then extends through the remainder of the line up to the carriage-return. Any valid ASCII characters are legal in the comments field.

3.2 DATA GENERATION STATEMENTS

Data generation statements resemble machine instructions in format and generate binary data within the program flow. The data generated is normally not interpreted during program run as executable instructions but rather as constant data such as ASCII messages to be typed or numeric values to be used by those instructions being executed. The general format of the data generation statement is:

```
{label:}  {operator} {operands} {;comments}
```

The label field is optional and follows the same format and rules as the machine instruction label field. The operator field contains the specific data generation mnemonic for the type of data desired. We discuss these codes in Chapter 5. The operands field contains the actual data to be generated by the statement and its format depends on the type of operator in use. Some operators such as WORD and BYTE allow multiple operands within the same statement so the amount of binary data generated by the statement is variable. If a label is used, its value is always that of the address into which the first byte of data will be assembled. The comments field is optional.

There is a default data generation statement you should be aware of. If no operator is present, M68 assumes the statement is a WORD statement—it skips the missing operator field and interprets the next field as the operands field. The assembler works in the following manner when analyzing statements:

1. Leading sybols terminated by colons are processed as labels and stored in the assembler symbol table.

2. The next symbol is first scanned for a match in the macro table which consists of all macros previously defined in the program.

3. If the operator symbol is not a macro name, it is then matched against the table of machine instruction opcodes, data generation operators, and assembly control pseudo opcodes.

4. If none of the above result in a defined operator, the default WORD processor is entered and the symbol is assumed to be the beginning of the associated operands field for the WORD statement.

3.3 SYMBOLIC EQUATE STATEMENTS

A user symbol may be assigned a value by following it on a statement line with an equal sign (=) and the expression it is to be equated to. The general format of the equate statement is:

```
{user symbol} = {expression}    {;comments}
```

The equal sign may have leading or trailing spaces and tabs if desired for formatting purposes. The expression may be any valid numeric expression but, since all equate statements must be fully resolved during Phase 1, any user symbols used in the expression must be defined at the time the assembler encounters the equate statement. Equate statements may not contain references to external symbols. The comments field is optional.

User symbols assigned values in the program may be reassigned a different value later in the program by using another equate statement to redefine the desired symbol. Labels may not be redefined by equate statements, however. If the relocation attribute of the evaluated expression is zero, the value assigned to the symbol is absolute. If the relocation attribute is non-zero, then the value assigned is relocatable. If the expression contains a register symbol, then the equated symbol is also given a register attribute. In other words, the value assigned to the user symbol pretty much follows the attributes of the expression to which it is equated.

3.4 ASSEMBLY CONTROL STATEMENTS

Assembly control statements cover a wide range of functions that generally set up or alter the parameters which control the assembly process. They do not themselves generate any binary code but are used for such purposes as listing format control, numeric radix assignment, and program generation or addressing information. The general format for assembly control statements is:

```
{pseudo-opcode} {arguments}      {;comments}
```

The pseudo opcode is the mnemonic that defines the function to be performed. Chapter 5 lists all assembly control pseudo opcodes with an explanation of what each one does. Some of them require arguments needed to set up parameters. These arguments are separated from the pseudo opcode by one or more spaces or tabs. The comments field is optional. Unless the explanation in Chapter 5 for a pseudo opcode specifies otherwise, labels are not normally permitted in assembly control statements.

3.5 CONDITIONAL ASSEMBLY DIRECTIVES

Conditional assembly is defined as selectively assembling or bypassing statements within defined bounds depending on the value of some variable at the time the assembly is performed. The bounds are made by conditional assembly directives which specify the variable or variables to be tested and the condition to be met in order for the assembly to occur. Conditional assembly directives are most commonly used in conjunction with macro definitions to direct the tailoring of each macro call as it is encountered. We discuss conditional assembly directives in Chapter 7.

3.6 MACRO DEFINITIONS AND MACRO CALLS

Macros are defined as one or more valid statements which may be called for by using a single symbol (the macro name) within the program anytime after the macro has been defined. Macros are always defined by you within your program or within a copy file which is called into your program by the COPY statement. See Chapter 5 for information on COPY files and UNIVERSAL files (a special form of the COPY file).

The COPY file called SYS.M68 is a macro library of over a hundred such macro definitions which define all the supervisor calls available to your programs for communicating with the monitor routines. This library file is supplied on the AMOS System disk in account [7,7].

Macro calls are those statements which name the defined macro as the operator of the statement and give the specific arguments to be used by the macro (if any are required). A macro call within the program causes the defined macro to be included in its tailored form at the point of the call. Macro calls normally cause one or more machine instructions to be assembled and binary code to be generated.

Chapter 6 defines macro definitions and macro calls more fully.

3.7 COMMENT LINES AND BLANK LINES

Statements which begin with a semicolon (after any leading spaces and tabs) are considered comment lines and do not result in the generation of any binary code or in the alteration of any assembly control parameters. They are useful for documenting the source programs and making them easier to read and maintain. Blank lines are also considered comment lines and are for appearances only in the source file. It is most important to fully document your programs, so be generous in your use of comments.

CHAPTER 4

TERMS AND EXPRESSIONS

This chapter describes the various terms and components used in M68 source statements, including the defined character set for the construction of symbols and expressions.

4.1 CHARACTER SET

The entire ASCII character set is legal in M68 source programs except for control-characters. Legal characters for a user symbol include the letters A-Z and a-z, the numbers 0-9, and the special characters "%," "." and "$". The list below gives the special characters recognized by the assembler when scanning source lines:

Special Characters

SYMBOL	MEANING
:	Label terminator.
;	Comment field indicator.
=	Equate statement operator.
#	Immediate expression indicator.
@	Deferred addressing indicator.
(	Initial register indicator.
)	Terminating register indicator.
,	Operand field or macro argument separator.
.	Value of the assembly current location counter when used as a term.

(Continued)

Special Characters (Continued)

SYMBOL	MEANING
<	Initial argument or expression indicator.
>	Terminating argument or expression indicator.
+	Arithmetic addition operator or autoincrement mode indicator.
–	Arithmetic subtraction operator or autodecrement mode indicator.
*	Arithmetic multiplication operator.
/	Arithmetic division operator.
&	Logical AND operator.
!	Logical inclusive OR operator.
’	Single ASCII character term indicator.
"	Double ASCII character term indicator.
[	Initial RAD50 term indicator.
]	Terminating RAD50 term indicator.
^	Universal unary indicator.
_	(Underscore) Binary shift operator.

The use of the above legal characters out of context for their designed purposes generates a syntax error (code Q).

4.2 TERMS

A term is the basic unit of data in an arithmetic expression and may be one of the following:

1. A number as composed of legal digits within the current numeric radix being used by the assembler at the time the term is assembled or as temporarily defined by the inclusion of a leading temporary radix change operator;

2. A user symbol (as previously defined in Chapter 3) which is given an assigned value either by its use as a label or a direct equate statement;

3. One or two ASCII characters defined by the single or double quote indicators;

4. A RAD50 triplet word or six-character longword enclosed in square brackets;

5. The period symbol (.) which represents the current value of the assembly current location counter;

6. An expression or term enclosed within angle brackets. Angle brackets are used to alter the normal hierarchy of expression evaluation, normally done in a left-to-right manner. Any quantity enclosed within angle brackets is evaluated before the remainder of the expression in which it is found. The action of angle brackets within a M68 source expression is the same as is used in the AlphaBASIC language. Angle brackets may also be used to apply a unary operator to an entire expression such as –<16/A>.

4.3 EXPRESSIONS

An expression is a combination of terms and operators which will evaluate to an unsigned 32-bit value in the decimal range of 0 through 4,294,967,295. Negative values in the range of –2,147,483,648 through –1 are stored properly after evaluation but are treated the same as their unsigned counterparts in the range of 2,147,483,648 through 4,294,967,295.

The evaluation of any expression also includes the evaluation of the mode of that expression (absolute, relocatable, or external) and the register designation of the expression.

Constants within numerical expressions are treated as SIGNED integers during expression evaluation. It is important you take this into account when forming arithemetic expressions. For example:

```
LWORD     ^HOFFFFFE00/8
```

yields ^HFFFFFFB7 instead of ^H249248BD because it is equivalent to:

```
LWORD     -512./7
```

Operators are defined as unary or binary. Unary operators precede a single term and alter the evaluation of that term alone. Multiple unary operators may be applied in sequence to the same term and are evaluated right to left. Binary operators combine two terms to give a resultant effective single term value. Multiple binary operators are illegal.

Expressions are evaluated left to right under the hierarchy of the operators in use within that expression. Angle brackets may be used to alter the normal process of evaluation.

Unary operators always take precedence over binary operators and are applied to the associated terms first.

Legal Operators

SYMBOL	OPERATION
+	Unary plus sign (default if term not preceded by another unary).
–	Unary minus sign which negates the associated term value.
^C	Unary one's complement operator (XORs the term with all ones).
_	(Underscore.) Binary shift operator. Given A_B, binary representation of A is shifted B number of times. If B is positive, shifts A left—if B is negative, shifts A right.
*	Binary multiplication operator.
/	Binary division operator.
+	Binary addition operator.
–	Binary subtraction operator.
&	Binary logical AND operator.
!	Binary logical inclusive OR operator.
^D	Temporary radix change to decimal for the associated term.
^B	Temporary radix change to binary for the associated term.
^O	Temporary radix change to octal for the associated term.
^H	Temporary radix change to hexadecimal for the associated term.

Two special operators (\ and ') also exist for use within macro definitions. See Chapter 6 for more information on operators.

Expressions are evaluated as being absolute, relocatable, or external. This distinction is important because code on Alpha Micro computers must be totally relocatable. The following rules apply in the evaluation of the relocation attribute of an expression:

1. An expression is absolute if its value is fixed and contains no relocatable terms. Also, a relocatable term minus another relocatable term results in an absolute value. Labels allocated within an absolute section (ASECT) are assigned absolute values and attributes.

2. An expression is relocatable if its value is fixed relative to the current program base which is relocatable at load time. The value may have an offset added to it by LNKLIT if it is not within the first segment of a program file. Labels allocated within a relocatable section (PSECT) are assigned relocatable values and attributes. For information on the ASECT and PSECT pseudo opcodes, see Chapter 5.

3. An expression is defined as external when one or more of its terms is an external symbol reference. This expression will not be fully resolved until the program file is generated by the linkage editor (LNKLIT), at which time the external terms are defined. The final resolution of an external expression may be relocatable or absolute, depending on the attributes of the terms involved (both internal and external). The linkage editor also contains all the mechanics for evaluating the attributes of resolved expressions.

See Chapter 5 for information on the EXTERN, INTERN, and AUTOEXTERN pseudo opcodes.

4.4 NUMBERS

Any source item which starts with a digit (0-9) is considered to be a number and this number is evaluated under the currently prevailing radix being used by the assembler at the time of evaluation unless preceded by a temporary radix operator or followed immediately by a decimal point. The prevailing radix always starts as octal (base 8) at the begining of any assembly but may be changed by the RADIX assembly control statement. Any number that terminates with a decimal point is evaluated as decimal (base 10) regardless of the prevailing radix. Fractional numbers are not allowed in M68 source statements since all numbers must evaluate to a 32-bit integer value.

The prevailing radix controls the default evaluation of numbers and may be set by the RADIX statement to any value from 2 (binary) through 36. Numbers in a base above 10 (decimal) use the letters A-Z to represent the digit values of 10 through 35. The most common system above base 10 is hexadecimal where the letters A-F represent the decimal digit values 10-15. All numbers must begin with a digit 0-9 to distinguish them from a user symbol, so the hexadecimal value of F56 must be entered as 0F56.

Negative numbers are preceded by a minus sign—M68 evaluates them and stores them in twos complement form. You may optionally precede positive numbers with a plus sign, but this is not required.

4.5 REGISTER SYMBOLS

The 680XX microprocessor (the heart of the Alpha Micro computer) contains sixteen 32-bit registers which are symbolically named and used as follows:

SYMBOL	REGISTER	SYMBOL	REGISTER
A0	Address register 0	D0	Data register 0
A1	Address register 1	D1	Data register 1
A2	Address register 2	D2	Data register 2
A3	Address register 2	D3	Data register 3
A4	Address register 4	D4	Data register 4
A5	Address register 5	D5	Data register 5
A6	Address register 6	D6	Data register 6
SP	Address register 7, stack pointer	D7	Data register 7

These sixteen symbols are already defined to the assembler and must be used when the address mode explicitly requires a register to be referenced. The above register symbols have a register attribute associated with them and you may equate your own symbols to these registers if you so desire. The register attribute is carried over to this newly defined symbol. For example, the equate statement IOPTR=D4 will equate the user symbol IOPTR to the value of 4 and also give it a register attribute so it may be used in place of D4 in address modes.

4.6 ASSEMBLY LOCATION COUNTER

During the assembly process, M68 assigns sequential memory locations to all machine instructions and data constants as it encounters them in the source program.

At any given statement, the next byte to be assigned is internally stored in the assembly location counter. This address may be used in expressions by referencing the period (.) as a symbolic term. For example, the instruction "JMP .+6" causes a jump to the address six bytes in front of the first byte of the JMP instruction itself. This method of addressing is very poor programming practice—it was originally provided to allow certain capabilities within macros otherwise not possible before macro labels were made available.

Initially, the value of the assembly location counter is set to zero and is incremented as each program statement which produces binary code is assembled during Phase 1 of the assembly. You may explicitly change the current location counter at any time by using a direct equate statement that uses the period symbol instead of a user symbol. For example, the statement ".=500" forces the assembly location counter to take on a value of 500 and to begin all following assembly allocations from that point.

4.6.1 Assembly Sections

Because the Alpha Micro computer is a time-sharing, multi-user computer, there is no way of knowing beforehand exactly what memory area a program will be loaded into when it is executed by a user. For that reason, assembly language programs on this computer must be "relocatable"—that is, they must be constructed so they can be loaded into any memory location and run without any modification. If a program contains code that must reside in specific memory locations, we say that it contains "absolute" rather than "relocatable" code.

To assist you in writing relocatable programs, the assembler assumes your program location counter is an offset to the starting address of the area of memory the program is ultimately loaded into. Thus, for example, if a user's memory partition starts at location 100000, and he or she loads your relocatable program into memory, program location #20 is loaded into memory location 100020, not location 000020.

However, there are special situations in which you may need to have more control over exactly how your program is loaded into memory. To give you this flexibility, M68 contains three pseudo opcodes, PSECT, ASECT, and DSECT. Each of these pseudo opcodes causes a different type of assembly section to be generated, each of which has its own assembly location counter.

PSECT

The PSECT opcode tells the assembler to generate a "program section"—that is, relocatable code. If your entire program is going to be relocatable, you do not need to use the PSECT opcode, since the assembler generates relocatable code by default. You will use the PSECT pseudo opcode to tell the assembler to resume generating a program section after you have used ASECT or DSECT.

Two other types of assembly sections exist—"absolute sections" and "data sections."

ASECT

The ASECT pseudo opcode tells the assembler the label definitions that follow are absolute (which means their addresses will not be adjusted by the LNKLIT program). Such an absolute section must only contain label definitions, and is used to define absolute memory locations such as system communication words and I/O addresses.

The ASECT location counter is reset to zero each time the assembler encounters an ASECT pseudo opcode.

DSECT

The DSECT pseudo opcode tells the assembler you wish to generate a "data section." A data section differs from a program section in that, while relocatable, it is specifically designed for declaring variables that will be placed in a separate memory module—an "impure area" where data for a particular user of the program can be maintained. This allows you to make the body of the program re-entrant (so it can be loaded into system memory where multiple users can share the single copy), while still keeping a data area for each user of the program in his or her own memory partition. The main advantage of the data section is that it allows you to declare in several different source files variables which are to be placed in a single impure module. When the assembled source files are linked together, all DSECT data areas are appended into a single data section.

Data sections may contain any type of assembler instruction or assembler pseudo opcode (with some exceptions mentioned later). However, when your program is processed by LNKLIT, the .LIT file does not contain any of the DSECT constants, definitions or pointers that you placed in the DSECT. Instead, a separate file is generated called an .RTI file. It contains information and data in a format similar to the .OBJ files produced by the assembler. The .RTI file is processed when your program executes the run time initializer (RTI). When RTI has initialized your DSECT (the impure zone), all the pointers, constants and definitions used in the DSECT at assembly time are in the impure zone.

DSECT initialization is not performed automatically. You must use the GETRTI monitor call at the beginning of your program (see the *AMOS Monitor Calls* for details). You must also specify the /D switch to LNKLIT or SYMLIT to generate the .RTI file. For details on how to use the /D switch, refer to Chapter 9 on the linkage editor (LNKLIT) and symbol table file generator (SYMLIT). That chapter also discusses the .RTI file in more detail. If you do not use the GETRTI call and the /D switch, your program's DSECT can only contain label definitions.

This AMOS facility is called DSECT initialization. The values may be resolved differently in the DSECT than in the PSECT. Here are some illustrations of how to use DSECT initialization.

```
        DSECT
QBLCK1: LWORD   QBLCK2
        BLKB    100.
QBLCK2: LWORD   QBLCK3
        BLKB    100.
QBLCK3: LWORD   0
        BLKB    100.
```

These statements set up a short queue of memory blocks. If this is done in the PSECT, the LWORD locations contain only the offset of the labels QBLCK2 and QBLCK3 in the PSECT. The program would have to resolve the actual memory locations of the queue elements and put the addresses in the LWORD locations. By using DSECT initialization and placing these statements in the DSECT, the LWORD locations contain the actual 32-bit memory addresses of the queue elements QBLCK2 and QBLCK3.

Pointers defined in the DSECT can point to other locations in the DSECT (as illustrated above) or they can point to locations in the PSECT.

```
        DSECT
PTR1:   LWORD   SUBR1
PTR2:   LWORD   SUBR2
PTR3:   LWORD   SUBR3
        PSECT
SUBR1:  AND     #7,D3
        ...
        ...
        ...
        RTN
SUBR2:  TST      D5
        ...
        ...
        ...
        RTN
SUBR3:  TYPECR   <Error encountered.>
        ...
        ...
        ...
        RTN
```

The example above shows a table of pointers in the DSECT which are pointing to subroutines in the PSECT. DSECT initialization sets the values of the pointers PTR1, PTR2, and PTR3 to the actual 32-bit memory addresses of the subroutines SUBR1, SUBR2, and SUBR3.

DSECT initialization can also be used to set up constants in the DSECT, allowing you to keep tables, arrays, and other data structures in the DSECT, leaving only the program code in the PSECT.

```
        DSECT
ASCTBL: BYTE    'A
        BYTE    'B
        BYTE    'C
        BYTE    'D
           ...
           ...
           ...
        BYTE    'Z
```

DSECT initialization would place the table illustrated above in the impure zone. DSECT initialization allows the use of BYTE statements, WORD statements, LWORD statements, ASCII statements, and RAD50 statements. Any type of statement that reserves memory with a constant value is valid.

If the instruction:

```
MOV     LABEL,TAG(A5)
```

is placed in the DSECT, the machine instruction generated contains the relative displacement of LABEL, not the actual 32-bit address. In other words, if you code machine instructions into the DSECT they are assembled exactly the same as if you had coded them in the PSECT.

Unless the offset value of a label exceeds the addressing limitations of the addressing mode "offset(An)" on your particular computer, your program should always access the data section by using the labels defined in it, indexed with a base register, containing the address of a memory module which has been allocated to hold the contents of the data section.

For example, if the symbol LINELN is defined as a word containing the length of a user's input, you might see if the line length is zero by using the following statement:

```
TSTW     LINELN(A0)      ; Is linelength zero?
```

where A0 is the base register.

The following pseudo opcodes are illegal in the DSECT:

```
DSIZE
OVRLAY
```

In order for your program to allocate the memory module that will contain all of the variables defined in a data section, it must know the total size of the data section. You must use the DSIZE pseudo opcode to find out how big a memory module to allocate. For information on DSIZE, see Chapter 5.

If you are limiting yourself to 16-bit offsets, a DSECT area ***cannot*** be larger than 32Kb in size. Addressing limits depend on several factors: whether the LNKLIT /X switch is used to allow a maximum DSECT size of 64Kb rather than 32Kb, the addressing mode you use, and the microprocessor used in your computer. For example, on a 68000-based computer, you cannot use the 32-bit specific addressing modes allowed on a 68020-based computer. (See Chapter 9 for information on the LNKLIT /X switch.)

Do not declare blank DSECT areas in your program. This may result in incorrect linkage of your .LIT file and produce errors during execution.

4.7 LOCAL SYMBOLS

M68 supports local symbols of the form nnn$, where nnn may be any number from 0 through 65535, decimal. A program using local symbols requires less symbol table space and assembles faster than a similar program without local symbols.

Local symbols of the form nnn$$ are used within macros and have scope within a particular macro expansion. For information on this kind of local symbol, see Chapter 6.

A local symbol only has scope between two non-local symbols. For example:

```
SEND:   MOVB    (A0)+,D1
        BEQ     10$
        TTY
        BR      SEND
10$     RTN

RCV:    KBD
        LEA     R0,BUF
10$     MOVB    (A2)+,(A0)+
        BNE     10$
        RTN

SUBR:   ...
```

10$ is defined twice in the program above. The first 10$ has a range from the definition of SEND up to but not including the definition of RCV. The second 10$ has a range from RCV up to SUBR.

You may also define local symbols with an equate (=).

It's usually a good idea to leave gaps in the numeric sequence of local symbols (e.g. 10$, 20$, 30$ ...) to allow for any future modifications to the program.

CHAPTER 5

ASSEMBLER PSEUDO OPCODES

A pseudo opcode is so named because although it looks much like a regular operation code, a pseudo opcode is not a true machine instruction and may or may not generate actual binary code. Pseudo opcodes are built into the assembler and provide a variety of useful functions that make the life of the programmer easier.

This chapter discusses the M68 pseudo opcodes available for your use. We classify the functions of the pseudo opcodes into three categories:

- Assembly control
- Data generation
- Segmentation

The sections below discuss each of these types of pseudo opcodes. Other chapters discuss several other pseudo opcodes used in special circumstances. For example, Chapter 6 discusses the pseudo opcodes you can use inside of macro definitions. For a full list of all pseudo opcodes, see the index.

5.1 ASSEMBLY CONTROL PSEUDO OPCODES

Assembly control statements do a variety of functions which don't in themselves generate any binary code but, instead, set up or alter certain parameters which control the assembly process. Each statement consists of a defined assembly control pseudo opcode followed by optional arguments as required by the specific format. The sections below discuss these pseudo opcodes and the arguments for each.

5.1.1 COPY

The COPY statement allows another file to be included in the assembled program at the point where the COPY statement is located. The entire copied file is assembled, but you may use conditional assembly statements to omit certain portions if you want to. The most common use of this statement is to include the standard copy file MAC:SYS.M68, which defines all system macros and system parameters. The COPY statement includes the specification of the file to be copied into the source program during assembly. For example:

```
COPY    DEF           ; My own set of macro definitions
                      ; in the file DEF.M68.
```

The actual source program is not modified—rather, the assembler gets the input from the copied file and then returns to the original source file as it assembles the source file. A copy file may not include another COPY statement within itself although the original file may include as many individual COPY statements as desired. The filespec may be a complete file specification defining a device and account. If you do not include an extension, M68 uses the default extension of .M68.

If you specify both a device and account, M68 looks for the copy file in the specified device and account. However, if you omit either a device or an account specification, M68 goes through several steps trying to find the specified file:

If you omit both the device and the account specification:

1. M68 looks for the file in the device and account you are logged into.

2. If the file does not exist in that account and if the source file is on a different device than the one you are logged into, M68 looks in the account you are logged into on the device containing the source file.

3. Finally, M68 looks in the System M68 account, DSK0:[7,7].

If you omit just the device specification:

1. M68 looks in the specified account on the device containing the source file.

2. If the file does not exist in that account, M68 looks in the specified account on the device you are logged into.

3. Finally, if the account specified is [7,7], M68 looks in the System M68 account, DSK0:[7,7].

If you omit just the account specification:

1. M68 looks in the account containing the source file on the specified device.

2. If the file does not exist in that account, and if the source file is in a different account than the one you are logged into, M68 looks on the specified device in the account you are logged into.

3. Finally, M68 looks in the System M68 account, DSK0:[7,7].

You may find it convenient to place copy files in the System M68 account, DSK0:[7,7], since they are then available to all programmers through the COPY statement.

M68 does not normally output the source statements in the copied file during the listing phase of the assembly since most users do not want the system copy file (SYS.M68) and other collections of common routines to be repeated in all program listings. You may override this by using a /L switch following the filespec in the statement—this includes the copied file in the assembly listing. For example:

```
COPY MYMAC.M68/L RETURN
```

As it assembles your program M68 reports any COPY statements encountered. For example:

```
Copying from DSK0:SYS.M68[7,7]
```

When you are assembling with the /N switch, the symbols defined in your program as well as the COPY file are not folded to upper case letters. For example, if you use the symbol "label" in your program and use the symbol "LABEL" in your COPY file, they are not processed as the same symbol by the assembler. If you do not use the /N switch, "label" and "LABEL" are considered the same symbol.

5.1.2 SEARCH

The SEARCH statement allows the symbols and macros from a universal (.UNV) file to be included in the assembly where the SEARCH statement occurs. A universal file is generated when you assemble a source file that contains the UNIVERSAL pseudo opcode. The source file that is to become a universal file must contain only macro and symbol definitions—it must not generate any code. See "UNIVERSAL" in this chapter for information on generating a .UNV file.

The advantage in using the SEARCH statement and a .UNV file rather than the COPY statement and a normal source file is because symbols and macros are loaded without assembling source text, a SEARCH operation is much faster than its equivalent COPY operation.

The most common use of this statement is for the inclusion of standard system symbol files such as SYS.UNV and SYSSYM.UNV that contain symbol and macro definitions of use to all programs. (These universal files reside in DSK0:[7,7] along with the system COPY file SYS.M68.) The SEARCH statement includes a specification of the file to be copied during the assembly. For example:

```
SEARCH  SYMS  ; My own set of symbol and macro definitions
```

The filespec may be a complete file specification defining a device and account. If you do not include an extension, M68 uses the default extension of .UNV.

The default search path when looking for the universal file is the same as that for the COPY statement (see "COPY" in this chapter for more information on the default search path. See "UNIVERSAL" for more information on .UNV files.)

As it assembles your program, M68 reports any SEARCH statements encountered. For example:

```
Searching DSK0:SYSSYM.UNV[7,7]
```

When you are assembling with the /N switch, the symbols defined in your program are not folded to upper case. However, if the .UNV file was assembled without the /N switch, the SEARCH command brings in only upper case symbols. The SYS.UNV file and the SYSSYM.UNV file are both assembled without the /N switch, so all the symbols in these .UNV files are folded to upper case. It is necessary when using symbols defined in SYS.UNV or SYSSYM.UNV to use only upper case letters in the source file.

5.1.3 OBJNAM

The OBJNAM pseudo opcode controls the names of output files produced by LNKLIT, SYMLIT, and M68. It tells these programs how you want to modify the output file name and extension. If you do not use OBJNAM, the programs M68, LNKLIT, and SYMLIT produce an output file with the same name as the input file and the appropriate extension. The format for OBJNAM is:

```
OBJNAM  file.ext
```

or:

```
OBJNAM  expr1{,...exprN}
```

where 1<=N<=3. That is, OBJNAM is followed by a filename and extension or by one to three expressions. If OBJNAM takes the second form, each expression is either 0 or a RAD50 value. The first expression denotes the first three characters of the filename, the second expression denotes the last three characters of the filename, and the third expression denotes the three characters of the file extension.

OBJNAM causes the output file names to be modified as follows (where you have specified "file" and "ext" in the OBJNAM statement line):

OBJNAM ARGUMENT	TYPE OF FILE	OUTPUT FILE
MYPROG.NEW	Object file	MYPROG.OBJ
MYPROG.NEW	Binary program file (usually .LIT)	MYPROG.NEW
MYPROG.NEW	Binary overlay file (usually .OVR)	MYPROG.NEW
MYPROG.NEW	Assembly listing file	MYPROG.LST
MYPROG.NEW	Load map file	MYPROG.MAP
MYPROG.NEW	Symbol table file	MYPROG.SYM

As you can see from the table above, if you include an extension in the OBJNAM statement, only the binary output file uses the specified extension. If you include a file name and omit the extension, M68 uses the extensions listed in the table above, and .LIT for a binary program file or .OVR for a binary overlay file.

If you use the expression list form of OBJNAM, and omit any of the expressions, or any of the expressions is zero, the corresponding portion of the source file name remains unmodified.

If you specify a file name of three characters, the last three characters of the original file name remain unchanged. If you specify less than or more than three characters in the file name, the entire original file name is replaced. For example, if you assemble the source file DEVCPY.M68, and use the OBJNAM statement:

```
OBJNAM   TEST
```

the assembled and linked output file has the name TEST.LIT. If you specify:

```
OBJNAM   TST
```

The assembled file is named TSTCPY.LIT. If you specify:

```
OBJNAM   TS
```

The assembled file is named TS.LIT.

5.1.4 PAGE

The PAGE statement causes your assembly listing to begin a new page before continuing with the listed output. No action takes place other than this during assembly.

5.1.5 LIST and NOLIST

You may obtain an assembly listing by using the /L assembly switch. The LIST and NOLIST pseudo opcodes control which portions of your program appear in the listing file. NOLIST disables listing, and LIST re-enables listing. The LIST and NOLIST pseudo opcodes do not appear in the listing.

M68 ignores the LIST and NOLIST pseudo opcodes if you use the optional /X assembly switch.

5.1.6 ASECT, PSECT and DSECT

The ASECT statement causes the assembler to process absolute label definitions. These label values are not modified during LNKLIT editing and the values assigned to labels do not have the relocatable attribute flag set.

The PSECT statement causes the assembler to generate code for the relocatable section of the program. This is the normal section for the AMOS computer, which always relocates the program in user memory. This code is modified during LNKLIT editing and the values assigned to labels have the relocatable attribute flag set.

The DSECT statement causes the assembler to generate a data area. This data section contains variables that are placed into a separate, impure memory module.

The assembler maintains three separate location counters for PSECT, ASECT, and DSECT assembly sections. For a more detailed discussion of the use of ASECT, DSECT, and PSECT, see Chapter 4.

5.1.7 DSIZE

When you use the DSECT pseudo opcode to generate a data section for your program, your program must be able to find out how large the total data section is. This is necessary so your program knows how large a memory module to allocate to hold the variables defined in the data section. The DSIZE pseudo opcode tells your program how large a module to allocate. For information on data sections, see Chapter 4.

DSIZE generates a longword in which LNKLIT returns the total size of the data section. When the /X (Extended DSECT) switch is used in the LNKLIT command line, the DSIZE value returned is the twos complement of the actual value.

You must only use DSIZE once in all of the files that make up a program, and you must not use DSIZE within an overlay.

The DSIZE statement takes the form:

```
DSIZE
```

and behaves like the data generation statement LWORD n, where ***n*** is the total size of the data section.

5.1.8 SYM and NOSYM

The SYM statement causes all following user symbols to be output to the object file along with their assigned values. These symbols are later used by the SYMLIT program to generate a reference file for the dynamic debugger program AlphaFIX. NOSYM inhibits this output for all following user symbols. The use of SYM and NOSYM does not cause any noticeable change in the actual program.

5.1.9 CREF, NOCREF and MAYCREF

To get a full cross reference listing, use the /R assembly switch. To see the cross reference listing on your terminal, use the /RT switch.

The three pseudo opcodes CREF, NOCREF, and MAYCREF control which portions of your program are processed in creating the cross reference.

CREF enables normal cross referencing. NOCREF suppresses all defined symols from the cross reference listing until M68 encounters a CREF or MAYCREF statement. MAYCREF tells M68 to suppress all symbols defined from the cross referrence listing if those symbols are never referenced.

For a full discussion of the format of the cross reference listing, see Chapter 8.

5.1.10 EVEN

The EVEN statement forces the next binary code to be generated on a word boundary (next even byte) by incrementing the assembly location counter if it is odd (no change if it is even). This is necessary since all instructions must lie on a word boundary for proper execution by the Alpha Micro computer. The format of the EVEN statement is:

```
EVEN
```

5.1.11 RADIX

The RADIX statement forces a new default radix to be set up in the assembler. The default radix of base 8 (octal) determines how all numbers not preceded by a temporary radix operator (^B, ^D, ^H, ^O) are interpreted if an explicit RADIX statement is not used. The RADIX format is:

```
RADIX    n
```

where the radix argument ***n*** must be a decimal number in the range of 2-36. Radix values above 10 use the letters A-Z to represent the digit values of 10-35 inclusively.

5.1.12 NVALU

M68 provides a parameterized assembly facility by allowing you to use the /V switch to specify a value on the M68 command line. The /V switch may take one of the forms listed in the table below.

The NVALU pseudo opcode allows your program to access the value specified in the /V assembly switch. The NVALU statement in your program takes the form:

```
NVALU    sym
```

which sets the symbol ***sym*** to one of the values below, depending on which /V switch was used

For each form of the /V switch, the table below shows what type of data may be included with the switch, and what the switch sets the NVALU symbol to.

SWITCH	:X	NVALU SYMBOL
/V:x	x is an octal or hex number (depending on the current radix being used by the assembler).	symbol is set to x.
/VO:x	x is an octal number.	Symbol is set to ^0x.
/VH:x	x is a hexadecimal number.	Symbol is set to ^H0x.
/VD:x	x is a decimal number.	Symbol is set to ^Dx.
/VA:x	x is one or two ASCII characters.	Symbol is set to ’x if a single ASCII character, "x if two ASCII characters.
/VR:x	x is one to three RAD50 characters.	Symbol is set to [x].

5.1.13 END

The END statement terminates the source file and is included only to give a defined end on the listing. In the absence of an END statement, the assembly terminates with the logical end of the input file. If an END statement is encountered anywhere in the source input, including inside a copied file, the assembly terminates whether the logical end of the input file has been reached or not.

As it assembles your program, M68 warns you if your program file does not contain an END statement:

```
Phase 1:    Missing END statement
```

5.1.14 ASMMSG

The ASMMSG statement displays a message during Phase 1 of the assembly. The message is a string of legal ASCII characters bounded on both ends by the same character, which must not be included in the message string itself. Any printing character may be used as a delimiter. Some examples are:

```
ASMMSG  /You are assembling MYPROG version 1.1/

IF      GE,TBLSIZ-256
        ASMMSG "Jump table too big (>255 entries)"
ENDC
```

An example of how ASMMSG displays messages during Phase 1 of the assembly:

```
Phase 1:        You are assembling MYPROG version 1.1
                Jump table too big (>255 entries)
                Work area: 10234 bytes, 936 used
Phase 2:        ...
```

5.1.15 ASMERP

The ASMERP statement behaves just like the ASMMSG statement discused above except it also generates an E error on the line containing the ASMERP statement.

5.2 DATA GENERATION PSEUDO OPCODES

The M68 assembler has several pseudo opcodes which generate specific data constants within the program area for use as text messages, constant values, tables, etc. This section lists these pseudo opcodes and gives details on the data formats they generate. All statements may have labels in which case the label is assigned the address that will receive the first byte of the generated data. All data statements begin allocating their specific data formats at the address specified by the assembly current location counter and generate multiple bytes in sequence, incrementing the current location counter as necessary. Those statements which generate byte data (BYTE, ASCII, BLKB) may begin and end on any byte address, odd or even. Those statements which generate word data (WORD, RAD50, BLKW, and BLKL) must begin on a word boundary (even byte) or a boundary error (B) results. The EVEN statement may be used at any point where the status of the current location counter is in doubt to insure an even boundary.

5.2.1 BYTE

The BYTE statement generates one or more bytes (eight bits each) of data. The arguments for generating the data consist of expressions separated by commas. Any legal expression is valid but only the lower byte is stored after evaluation. Some examples are:

```
ZER:    BYTE  0               ; Generates 1 byte of data containing zero
        BYTE  1,2,3           ; Generates 3 bytes of data containing 1,2,3
MULTI:  BYTE  A-B,TAG*4,SAM   ; Generates 3 bytes of data
        BYTE  'A,'Q           ; Generates 2 bytes of ASCII data
```

BYTE checks that the argument is in the range -255 to 255 decimal.

5.2.2 WORD

The WORD statement generates one or more words (16 bits each) of data. The arguments for generating the data consist of expressions separated by commas. Any legal expression which evaluates into a 16-bit value is valid. WORD statements are generated by default if the first symbol on a line (after any labels) is not defined as an opcode, pseudo opcode or macro name. Some examples are:

```
ZER:    WORD   0              ; Generates 1 word (2 bytes) of data zero
        WORD   1,2,3          ; Generates 3 words of data containing 1,2,3
        WORD   A-B,"QT,SAM-.  ; Generates 3 words of data
        SAM                   ; Generates by default the value of SAM
```

WORD checks that the argument is in the range -65535 to 65535 decimal.

5.2.3 WORD32

The WORD32 statement causes a restricted WORD size of from -32768 to 32767 to be used. The combination of WORD32 and WORD64 can aid you in preventing unnoticed word size overflows in large programs. It is also useful for jump tables that access routines in the forward and backward directions. For example:

```
         WORD32
JPTBL:   OFFSET    RTI
         OFFSET    RT2
         OFFSET    RT3
         WORD      -1
         WORD64
```

The WORD32 statement insures the data words generated by the OFFSET macro don't exceed the PC indexed offset value of +-32K. Errors are generated when the distance offset to the routines exceed this 32K boundary. WORD64 then disables this constricted range.

5.2.4 WORD64

The WORD64 statement causes the default WORD size (see word above). It is used to reset normal WORD mode after a WORD32 statement is used.

5.2.5 LWORD

The LWORD statement generates one or more words (32 bits each) of data. The arguments for generating the data consist of expressions separated by commas. Any legal expression which evaluates into a 32-bit value is valid. An example is:

```
ZER:   LWORD  0     ; Generates 1 longword (4 bytes) of data zero
```

5.2.6∞ASCII

The ASCII statement generates one or more bytes of ASCII data. The argument for generating the data is a string of legal ASCII characters bounded on both ends by the same character which must not be included in the data string itself. Any printing character may be used as a delimiter. Some examples are:

```
MSG:   ASCII  /THIS IS A MESSAGE/  ; Generates a string of 17 data bytes
       ASCII  /Q/                  ; Generates a single data byte of "Q"
MSG2:  ASCII  $ I/O TERM $         ; Generates a string of 10 data bytes
```

5.2.7∞ASCIZ

The ASCIZ statement is similar to the ASCII statement. It also generates one or more bytes of ASCII data. However, it follows that data with a zero byte. For example, the following statement:

```
ASCIZ   /This is a message/
```

is equivalent to the two statements:

```
ASCII   /This is a message/
BYTE    0
```

5.2.8∞RAD50

The RAD50 statement generates one or more words (16 bits each) of data. The argument is a string of valid RAD50-packable characters bounded on both ends by the same character which must not be included in the data string. Any printing character may be used as a delimiter. The legal characters for RAD50 packing are A-Z, 0-9, dollar-sign ($), period (.), percent sign (%), and space. One packed word is generated for each three characters in the string or fraction thereof, with trailing spaces being assumed to fill out the last triplet. Some examples are:

```
DDB:    RAD50   /DSK/           ; Generates one word of packed data
        RAD50   /SAM QQ/        ; Generates two words of packed data
        RAD50   /ABCD/          ; Generates two words
                                ;   (same as RAD50 /ABCD°°/)
```

5.2.9∞OFFSET

There are many times during the programming of totally relocatable code in which an address must be expressed and stored as a relative offset from the location of the constant itself. In other words, the storage of the address TAG must be in the form of TAG–. which is actually the offset from the current position of the constant to the address defined as TAG. The value of this constant offset will not change no matter what its position in memory happens to be. The OFFSET pseudo opcode has been implemented to make the listings a little more obvious in intent. The OFFSET opcode

takes a single address argument and generates the relative offset to that address from the current position of the constant.

5.2.10 BLKB, BLKW, BLKL

These statements do not actually generate data but are included in this section because they result in the allocation of memory in a defined manner. BLKB allocates an area of bytes, BLKW allocates an area of words, and BLKL generates an area of longwords. In all other respects, they operate the same.

The argument for each is a single expression which evaluates to a value between 0 and 65535. This value is then added to the assembly current location counter (twice if BLKW, four times if BLKL) which effectively reserves that block of memory and continues allocating memory at the new address. Normally this results in a contiguous area of all zeros since the linker clears all blank areas when it generates the program file. This action does not always happen, however, because the location counter may be stepped back into the reserved area in which case the new data overlays the reserved block of memory. This is an important concept in dealing with the absolute section, since no data is actually generated by these statements—only memory addresses are reserved. Some examples are:

```
DATA:   BLKB    44      ; Reserves 44 bytes of memory
        BLKB    A*B     ; Reserves A*B bytes of memory
        BLKW    200     ; Reserves 200 words (400 bytes) of memory
        BLKL    10      ; Reserves 10 longwords (40 bytes)
```

5.3 SEGMENTATION PSEUDO OPCODES

The M68 assembler, together with the LNKLIT editor and monitor overlay calls, support a powerful method of segmenting and overlaying programs for convenience during system development and memory conservation during execution. This section describes the methods available for the various options and also the assembler pseudo opcodes which help support the system. The pseudo opcodes we discuss are AUTOEXTERN, INTERN, EXTERN, UNIVERSAL, and OVRLAY. This section also briefly discusses the concept of program libraries.

5.3.1 Segmenting Assembly Language Programs

There are several reasons for segmenting a program and also different methods for doing so, depending on the end result desired. A very large source program takes longer to edit (even a small change) and gives a greater opportunity for total loss if some disaster strikes the file links. A large program also takes longer to assemble and more memory to do so. Segmented programs may be organized to allow portions of the program to be resident in memory and other portions to be called in from disk only as required. Segmented programs may also contain duplicate symbols if the program segments are assembled separately and linked together by LNKLIT. Also, program segments which are assembled separately may be listed separately, resulting in less listing time and less paper used for each change made.

COPY Files

The simplest method for creating a program in segments provides one of the above advantages. This method uses the COPY statement and allows a large program to be edited as multiple segments which are then copied into the main source program by using one COPY statement for each segment. As changes are made to the source program, you need only edit the segment which requires the changes. The assembly is done, however, on the complete source program since all copied files are included in the source input. Only one object file results and only one single list file can be created. The /L option to COPY controls what segments are included on the listing.

Multiple .OBJ Files

A more complex but flexible method is to break the program up into logical segments which may be assembled separately and linked together at a later time by LNKLIT. The different segment assemblies result in several object (.OBJ) files—they are then input to LNKLIT, which creates a fully resolved and runnable program (.LIT) file. This method gives the advantages of using COPY as well as the added advantage of having to assemble only those segments which require changes. The LNKLIT process runs much faster and requires less user memory than the assembly process.

One of the requirements of a program which is segmented in this manner is that all references to routines and data constants which reside in another segment must be done through two special assembler pseudo opcodes, INTERN and EXTERN. Since a reference to a routine in another segment is not defined during the assembly of the calling segment, the symbol (name of the routine) is said to be "external." It is declared external by the EXTERN statement which tells the assembler it is defined and will be resolved by LNKLIT at a later time. The segment in which the routine exists then declares that symbol as "internal" via the INTERN statement which tells the assembler to output the symbol with a special code which defines it to LNKLIT for final resolution.

Overlays

The method of segmenting a program and then creating a single runnable program with LNKLIT may be extended one step further using a feature in the monitor which lets program segments be called in from the disk to overlay an existing portion of the main program. A segment to be used as an overlay defines itself as such by using the OVRLAY statement and giving the address at which the overlay is to be loaded. The main program then uses a special form of the FETCH supervisor call to load the overlay segment and executes it by jumping to a known segment start address. This method of overlaying segments is used in the M68 assembler itself and conserves user memory during execution of large system programs. LNKLIT creates one program (.LIT) file for the main segment and one overlay (.OVR) file for each overlay segment in use.

Object File Libraries

Still another method for modularizing programs is the use of library files. Program libraries allow you to use frequently used routines in many different programs without rewriting those routines each time you need them.

You may specify one or more library (.LIB) files to LNKLIT, which then links in from the .LIB file only those object files necessary to resolve external references. For full information on generating and maintaining program library files, see Chapter 10.

5.3.2 AUTOEXTERN

The AUTOEXTERN pseudo opcode tells M68 to automatically EXTERN any undefined symbols—those symbols are then displayed at the end of Phase 2 of the assembly. When AUTOEXTERN is in effect you do not have to explicitly EXTERN symbols.

5.3.3 INTERN

The INTERN statement defines one or more user symbols as internal to the program segment so they will be defined to LNKLIT for final resolution. The INTERN statement takes the form:

```
INTERN  sym1{,sym2,...symN}
```

Each INTERN statement may be followed by one or more internal user symbols separated by commas. As many INTERN statements as required may be used in the program. There is also no limit to the number of symbols each INTERN statement may reference except for the physical line length.

Each symbol referenced in an INTERN statement must be defined within the segment either as a label on a routine or constant or as a value by an equate statement. The symbol will then be available to LNKLIT for resolving references to it which come from EXTERN statements in other segments. Any symbol defined as external in a segment and not defined as internal in another segment results in an undefined error during linkage editing. A symbol may never be defined by more than one INTERN statement during any one LNKLIT run—i.e., the same symbol cannot appear as internal in two different segments that will eventually be linked into the same program.

A shorthand notation for INTERNing a label or equated symbol exists. Instead of writing:

```
INTERN  Symbol
Symbol:
```

you may write:

```
Symbol::
```

Instead of writing:

```
INTERN  Symbol
Symbol = Expression
```

you may write:

```
Symbol == Expression
```

5.3.4 EXTERN

The EXTERN statement defines one or more user symbols as external to the segment so they may be resolved by LNKLIT. The EXTERN statement takes the form:

```
EXTERN  sym1{,sym2,...symN}
```

Each EXTERN statement contain one or more user symbols separated by commas. As many EXTERN statements as required may be used in the program. There is also no limit to the number of symbols each EXTERN statement may define except for the physical line length.

Each symbol defined by an EXTERN statement may be referenced within the segment just as if it had been defined within the segment as a label or an equate statement item. There is no limitation placed on its use as a term within any operand expression since LNKLIT has complete expression resolution mechanics built in. There are two restrictions to its use within the segment. First, an externally defined symbol may not be used within the address operand of any branch instructions (BR, BEQ, BGT, etc.) because there is no way to insure the resulting placement will fall within the 127-word relative requirement. It may, however, be used within the address operand of the jump (JMP) instruction. The second restriction is, an equate statement may not contain any externally defined symbols in its operand expression since all equates must be fully resolvable as they are encountered.

LNKLIT builds a symbol table from all the symbols referenced in all INTERN statements in all program segments. It then goes back and resolves all expressions containing symbols defined by EXTERN statements by looking them up in the table of INTERN symbols. Any symbol defined in an EXTERN statement but not matched by some INTERN symbol gives an error message during linkage editing.

5.3.5 UNIVERSAL

The UNIVERSAL statement tells M68 you want to create a universal (.UNV) file to be used with the SEARCH statement. Universal files may only contain symbol and macro definitions—they must generate no code. When the UNIVERSAL statement is encountered, M68 knows you want to generate a .UNV file as output rather than the normal .OBJ files.

A typical example of the use of the UNIVERSAL statement is the system definition file, SYS.M68 which, when assembled, produces the SYS.UNV file.

Files being assembled into universal files may not include COPY or SEARCH statements. If M68 encounters a UNIVERSAL statement during a COPY operation, such as when SYS.M68 is copied, it ignores the UNIVERSAL statement.

Only when you are assembling the source file by itself does M68 create the .UNV file. This allows one source module to generate a universal file to be used with the SEARCH operation, and also allows the source file to be used with the COPY statement. Source files that contain the UNIVERSAL statement are not required to have an END statement for the same reason. For information on the SEARCH statement, see "SEARCH," in this chapter.

5.3.6 OVRLAY

The OVRLAY statement identifies a program segment as being an overlay file instead of a continuation of the main program file. It also defines the address of the base of the overlay relative to the base of the main program so the loading of the overlay segment is done at the proper spot in the program memory area. The OVRLAY statement takes a single argument which is a user symbol that must be defined in some other segment in an INTERN statement. For example:

```
OVRLAY   Sym
```

It is legal to write:

```
         OVRLAY   Sym
Sym:     ...
```

so long as ***sym:*** appears at the start of the overlay (the symbol "sym" is essentially defined twice with the same value). The OVRLAY address is resolved by LNKLIT when the files are processed. For more information on processing of the OVRLAY statement, see Chapter 9. For information on the code used to load the overlay segments into memory, see the description of the FETCH monitor call in the *AMOS Monitor Calls* manual.

CHAPTER 6

USER DEFINED MACROS

It is often convenient to create your own opcode definitions. When you use these definitions in your source program, they result in a sequence of one or more source code statements. These user-created opcodes are called "macros" in assembly language programming and the Alpha Micro assembler supports a flexible macro subsystem.

There are two phases you go through when using macro calls. First, you define the macro opcode once in the program as a series of source code statements along with possible dummy arguments. You only do this once—the macro remains defined throughout the remainder of the assembly process. Second, you invoke the macro by a single source statement giving the macro name along with optional real arguments that replace the defined dummy arguments in the macro source code which is generated. Calling the macro in this mannner replaces the macro statement with the defined sequence of source code statements custom tailored by the optional real arguments in the calling statement. You may perform this calling sequence as many times as needed in the source program with as many different real arguments as desired.

6.1 MACRO DEFINITION

Defining a macro generates no actual binary code in the program—it merely places the macro definition in a special table in the assembler memory work area. Calling the macro is the process that actually generates the binary code. If your program never calls the macro or if the macro does not contain any code-generating source statements, M68 produces no binary code for the macro. The use of conditional assembly directives within a macro definition may result in no code-generating statements for this particular call to the macro. The fact that no code is actually generated if the macro is never called is an important concept since it allows you to create macro libraries containing many macro definitions standard for a particular user computer. Those macros that are never called in any specific program do not generate any code and therefore take up no additional memory. The system library SYS.M68 contains over a hundred such macro definitions that define the supervisor calls to the monitor. For information on the system macros, see the *AMOS Monitor Calls* manual.

6.1.1 Macro Definition Formats

There are two formats available for defining macros. The normal format allows one or more source lines to be generated as a result of the macro call. The single-line format restricts the macro definition to one line of generated source code but takes up less room in the source listing. For several sample macros, see below.

The general format for multiple-line macros is:

```
DEFINE  name    {dummy argument list}
        source line 1
        source line 2
        ...
        ...
        source line n
        ENDM
```

The general format for a single-line macro is:

```
DEFINE  name    {dummy argument list} = source line
```

In both forms above, the macro name is any legal user symbol—it effectively becomes the opcode by which the macro is called. This symbol may duplicate a label in the program or may even redefine a pseudo opcode or machine opcode (e.g., you can redefine the MOV opcode to do an ADD if you really want to confuse people). You may only define a macro name once and an attempt to redefine it later in the program will give unspecified results.

6.1.2 The Macro Source Statements

The multiple-line macro definition source statements begin with the line immediately following the DEFINE statement and continue to but do not include the terminating ENDM statement.

Every macro definition must end with the ENDM pseudo opcode.

When the program text calls the macro, M68 generates and assembles all macro source lines just as if they had been explicitly entered into the source program.

In the single-line form, the source line begins with the character following the equal sign and continues through (and including) the carriage-return and line-feed which terminate the DEFINE statement line.

Macro definitions must not be nested within other macro definitions. Macro processing is done on a special prepass scheme which prohibits the processing of any DEFINE statements within another DEFINE statement.

6.1.3 The Dummy Argument List

The dummy argument list is optional in both forms of macros and consists of one or more user symbols separated by commas. These symbols are unique only within the actual definition of the current macro and may be duplicated in other macro argument lists or may even be other opcodes and defined symbols.

Dummy argument symbols never appear as such in the generated sequence of source statements when the macro is called but are replaced by the equivalent real arguments supplied in the calling statement. The dummy argument symbols may appear anywhere in the definiton source lines, even as labels. Each time M68 encounters a dummy argument when generating the source lines during a macro call, it replaces the dummy argument with the corresponding real argument supplied by the calling statement.

6.1.4 Labels

A label must not be used on the DEFINE statement line, since it has no meaning. Labels may be used in the calling statements. A label must not be used on the ENDM line or the assembler will not detect the ENDM line.

6.1.5 Local Symbols

M68 supports local symbols of the form nnn$ and nnn$$, where nnn is a number between 0 and 65535, decimal. Local symbols of the form nnn$ have scope only between two non-local labels, and may be used outside of macro definitions,.

Local symbols of the form nnn$$ are for use only within macro definitions. If a nnn$$ label appears outside of a macro, M68 will treat the label like nnn$ except the label will not appear in the symbol table file (used for debugging purposes). You may define a local symbol with an equate (=).

Below are two sample macros that use local symbols:

```
DEFINE  LEAMSG  X
        LEA     A0,10$$ ; Get address of message
        BR      20$$    ; Branch around message
10$$:   ASCII   'X'
        BYTE    0
        EVEN
20$$:
        ENDM
```

Now we call the macro:

```
LEAMSG  HELLO
TTYL    @A0             ; Display HELLO
LEAMSG  BYE
TTYL    @A0             ; Display BYE
```

The example above works correctly even though it generates two occurrences of 10$$ and 20$$ because the symbols are local to each macro call.

The example below demonstrates that local labels of the form nnn$ can be passed as arguments to macros, and that they are distinguished from labels of the form nnn$$ even if "nnn" is the same:

```
DEFINE  JGT10   X,Y
        CMP     X,#10
        BLE     1$$
        JMP     Y
1$$:
        ENDM
```

Now we call the macro:

```
        JGT10   D0,$1   ; Expands to:
                        ;     CMP   DO,#10
                        ;     BLE   1$$
                        ;     JMP   1$
                        ; 1$$:
        DEC     D0
1$:     RTN
```

6.1.6 Comments

A comment may follow the dummy argument list in the multiple-line form but you should not use a comment with the single-line form. You should avoid comments in the actual generated source lines in the macro definition simply because M68 stores the entire source text in work memory as ASCII characters (including all comments). This may use up so much work memory you might not have enough memory to finish the assembly.

6.1.7 Special Macro Operators

Two special operators are used only within macro definitions—the argument concatenation operator (’) and the expression evaluation operator (\).

Argument Concatenation (’)

Since dummy arguments must be valid user symbols, the apostrophe (’) is a legal delimiter for any dummy argument within a macro definition source line. When an apostrophe immediately precedes and/or follows a dummy argument in the source text, the apostrophe is removed and the substitution of the real argument occurs at that point.

This is useful for building symbols with arguments that are to be a part of that symbol.

Given the following macro defintion and calls:

```
DEFINE  BUILD   AA,BB
TAG'AA: MOV     D1,Q'BB'7
        ENDM
        ...
        ...
        BUILD   RA,STS
        BUILD   T,P
```

The effective code generated by the two calls would be:

```
TAGRA:  MOV     D1,QSTS7
TAGT:   MOV     D1,QP7
```

Expression Evaluation (\)

The \ operator tells M68 to evaluate the expression that follows and return its value. Within a macro definition you may use an expression of the form:

```
\expr
```

M68 then evaluates the expression and returns its value as a string. By placing a symbol in front of the \, you can direct M68 to append the value of the expression following the \ on the end of the symbol. For example:

```
LABEL\4*4:
```

evaluates to:

```
LABEL16:
```

and:

```
$ = 1
STC\$:
```

evaluates to:

```
STC1:
```

Symbols generated in this way do take up room in the symbol table.

Be very careful that the expression following the \ operator does not contain any macro argument—it will not be expanded properly and will probably cause a syntax error (Q code).

6.1.8 Suppressing Macro Expansion - ENDMX

The ENDMX pseudo opcode ends the expansion of the current macro. This pseudo opcode is illegal outside of a macro definition. You will find this pseudo opcode useful when using conditional assembly directive pseudo opcodes to control macro expansion.

ENDMX controls what macro code is generated at the time of a macro call—it does not affect whether the macro expansion is included in your assembly listing.

6.1.9 NCHR, NTYPE, NEVAL and NSIZE

These four macro directives return a value that specifies the number of characters in an argument (NCHR), the addressing mode type of an argument (NTYPE), the value of any extra word generated by the addressing mode evaluation (NEVAL), or the length of any extra words generated by an addressing mode (NSIZE).

These statements function similarly to the equate statement (=) in that they assign a value to a user symbol which may be reassigned as many times as desired during the course of the assembly. They are normally used to control the development of macro source code based on the size and type of arguments passed to the macro and therefore are defined in this section dealing with macros. In actuality, you may use them anywhere in the source program with any valid source code as an argument but they are fairly meaningless unless used within a macro.

Once the symbol has been assigned a value by one of these directives, you may use it by itself or within expressions to control the development of the macro source code through the conditional assembly statements.

NCHR

The NCHR statement assigns the value of the number of characters in the argument string to a user symbol. It has the format:

```
NCHR    symbol, string
```

NTYPE

The NTYPE statement assigns the value of the 6-bit addressing mode of the argument to a user symbol. It has the format:

```
NTYPE   symbol, argument
```

The following is a list of the addressing modes and the values they deliver via the NTYPE statement. The upper case "D" represents any data register (D0-D7), while an upper case "A" stands for address registers (A0-A7), including SP. The upper case "R" signifies any register in general.

MODE	NTYPE RETURNS
Dn	Direct register delivers On
An	Direct register delivers 1n
@An	Indirect address register delivers 2n
(An)+	Post-increment address register indirect delivers 3n
-(An)	Pre-decrement address register indirect delivers 4n
x(An)	Address register indirect with offset delivers 5n
x(An)[Rm]	Indexed register indirect with offset delivers 6n
xxxx	Absolute word address delivers 70
xxxxxxxx	Absolute longword address delivers 71
x	PC-relative delivers 72
x[Rm]	PC-relative indexed delivers 73
#x	Immediate delivers 74

For example, if you use register A4 in indirect addressing mode @A4, NTYPE returns a 24 (i.e., 2n where n = register A4).

When doing an NTYPE on absolute word, absolute longword, PC relative, and PC relative indexed (address modes 70, 71, 72, and 73 respectively), the symbol being NTYPEd ***must*** be defined in the program source file before the NTYPE statement is used. NTYPE pseudo-opcodes are resolved in phase one of the assembler only. If the symbol being NTYPEd is defined after the NTYPE statement is used, or if the symbol is an externally defined symbol, the NTYPE statement returns a value of 70.

NEVAL

The NEVAL statement assigns to a user symbol the value of the extra word generated by one of the indexed, relative or immediate addressing modes. This word represents the index argument for indexed modes, the relative offset for relative modes or the immediate value for the immediate mode. It has the format:

```
NEVAL   symbol, argument
```

NSIZE

The NSIZE statement assigns the value of the size of the address form to a user symbol (i.e., 0 if no extra word is generated, 2 if an extra word is generated). It has the format:

```
NSIZE   symbol, argument
```

6.1.10 Sample Macro Definitions

Below are several sample macro definitions.

A macro called ADDIT which generates four instructions:

```
DEFINE  ADDIT
        MOV     D1,D3
        ADD     D3,SUM
        ASL     D3
        ADD     D3,SUM
        ENDM
```

A macro called XCHNG which exchanges two memory words:

```
DEFINE  XCHNG   MEMA,MEMB
        MOV     MEMA,D1
        MOV     MEMB,MEMA
        MOV     D1,MEMB
        ENDM
```

A macro called STKSUB which subtracts a memory word from the top stack word:

```
DEFINE  STKSUB  TAG
        SUBW    TAG,@SP
        ENDM
```

The same STKSUB macro in the single-line format:

```
DEFINE  STKSUB   TAG = SUBW   TAG,@SP
```

For more complex examples of macro definitions, print out or inspect the system macro library SYS.M68 that defines all of the supervisor calls used by the Alpha Micro computer.

6.2 MACRO CALLS

The actual generation of the defined source code comes when you call the macro by its name within the text of your source program. The macro must have been defined prior to its first reference. Macros are only processed for definition during Phase 1 of the assembly process. Macro calls have the same format regardless of whether the macro definition is multiple or single line format:

```
{label:} name  {real arguments}       {;comments}
```

6.2.1 Name

Name represents the name given to the macro definition—this becomes the effective opcode by which program calls the macro.

6.2.2 Real Arguments

Use real arguments when the definition of the macro has a dummy argument list—they replace the dummy arguments in the source code text of the macro definition. The real arguments replace the dummy arguments on a one-for-one basis in exactly the same order as the elements of the dummy argument list. The first real argument in the call takes the place of each occurrence of the first dummy argument in the definition, and so on for all the arguments. If there are not enough real arguments given in the call to fill all required dummy arguments, the unfilled dummy arguments take on a null value and are effectively replaced with nothing. If there are more arguments in the call than required to fill the dummy arguments in the definition, M68 ignores the excess arguments.

Real Argument Format

Normally, the real arguments are separated by commas and the assembler expects this format. Also, leading and trailing blanks are ignored when processing each real argument in the macro call statement. Often you may want to include a comma or blank as part of the real argument without having it act as a delimiter or be bypassed. Any argument enclosed in angle brackets is passed onto the source code generation verbatim, including any blanks and commas.

The macro call:

```
XPURT ONE,TWO,THREE
```

has three real arguments while the call:

```
XPURT <ONE,TWO,THREE>
```

has only one argument which includes the two commas. The call:

```
XPURT <ONE,TWO>,THREE
```

has two real arguments of which the first includes one comma.

The system macro TYPE is another good example of using dummy arguments:

```
DEFINE  TYPE    MSG
        TTYI
        ASCII   /MSG/
        BYTE    0
        EVEN
        ENDM
```

This macro is one of the AMOS monitor calls and types out the ASCII message which appears as the argument to the TYPE macro call. The BYTE 0 statement insures a null terminator and the EVEN statement insures the next instruction is again synchronized on a word boundary.

The call:

```
TYPE  HELLO
```

types the message "HELLO" because all the leading blanks are automatically ignored before the argument is processed. The call:

```
TYPE  <°°°°°HELLO°>
```

types the message "°°°°°HELLO°" because the blanks are included in the argument as a result of the angle brackets. Similarly, the call:

```
TYPE  HELLO, I AM A COMPUTER
```

types the message "HELLO" because the comma will terminate the argument and the rest of it is ignored. The call:

```
TYPE  <HELLO, I AM A COMPUTER>
```

types the message "HELLO, I AM A COMPUTER" because the comma is included in the argument as a result of the angle brackets.

6.2.3°Label

The label is optional and is assigned the address contained by the assembly current location counter. This is normally the address of the first byte of code which is generated by the macro source lines (assuming the macro does actually generate code). If the macro does not generate code, then the label is still defined but it represents the address of the next byte of code generated after the macro call.

6.2.4 Comments

Ccomments are optional.

6.2.5 Nested Macro Calls

Macro calls may be nested to a depth of 32 levels. A nested macro is defined as a macro call within the source statements generated by another macro call. Arguments may be passed to nested macros by naming the dummy arguments the same throughout the levels.

Arguments that contain blanks or commas may be passed through successive levels by enclosing them in one set of angle brackets for each level of nesting since one set of angle brackets is removed from an argument with each nesting level. For example, to pass the argument A,B through three levels of nested macro calls, enter the argument as <<<A,B>>> in the first level macro call.

6.2.6 Sample Macro Calls

Consider this example:

```
DEFINE  TBLADD  ARG1,ARG2,ARG3
        MOV     ARG1,D1
        ADD     ARG2,D1
        MOV     D1,ARG1,(ARG3)
        ENDM
```

This macro is called TBLADD and requires three real arguments. Assume the following call in your program:

```
SAM:    TBLADD  SUMS,ENTRY,R5
```

The following source statements are generated:

```
SAM:    MOV     SUMS,D1
        ADD     ENTRY,D1
        MOV     D1,SUMS(A5)
```

It is evident from its usage that ARG3 must be a register. Assume only two arguments were given in the call:

```
SAM:    TBLADD  SUMS,ENTRY
```

The following source statements are generated:

```
SAM:    MOV     SUMS,D1
        ADD     ENTRY,D1
        MOV     D1,SUMS()
```

Notice the third instruction contains an error because of the missing register term resulting from the missing third argument. Sometimes a missing argument may be used to advantage by altering the generation of the source statements with the conditional assembly statements. These statements (described in the next chapter) can detect that the argument is missing and can be used to selectively omit portions of code.

CHAPTER 7

CONDITIONAL ASSEMBLY DIRECTIVES

Conditional assembly directives allow you to selectively include or bypass certain lines or segments of source code based on variable parameters which are tested during assembly. This allows several different versions of the same program to be generated from one source file. Conditional assembly directives find their widest use within macro definitions where they are used to tailor the macro based on the real arguments used in the macro call.

7.1 CONDITIONAL DIRECTIVE FORMATS

Like the macro definitions, conditional directives follow two general forms. The normal form allows one or more lines of source code to be selected or bypassed based on the current status of a variable. The single line form performs the same function but only allows the control of a single line of source code.

The general form of a normal conditional block is:

```
IF      condition,argument
        source line 1
        source line 2
        ...
        ...
        source line n
        ENDC
```

The general form of a single-line conditional is:

```
IF      condition,argument, source-line
```

Both forms use the IF pseudo opcode to identify the conditional directive and both forms require a condition code which specifies the type of test to perform and an argument on which to perform that test. The condition code is a symbol which identifies the test performed when the conditional is encountered during Phase 1 of the assembly process. The argument may be a symbol, expression, or macro argument, depending on the type of test being performed.

The item that distinguishes the two forms is the comma following the argument in the single-line form. If the comma exists, the remainder of the line up to and including the carriage return/line-feed is the source line that will either be assembled or bypassed depending on the result of the conditional test. If the comma does not exist, the conditional assembly will be done on the source lines that follow the conditional directive IF line up to but not including the ENDC terminating line.

7.2 CONDITION CODES

The following is a list of valid condition codes and the conditions the associated argument is tested for. Unless otherwise specified, the argument is evaluated as an expression and the 16-bit result of that evaluation is tested to meet the condition. The conditional source lines are assembled if the argument meets the condition listed next to the code below.

CODE	CONDITION
EQ	Argument is equal to zero.
NE	Argument is not equal to zero.
LT	Argument is less than zero.
GT	Argument is greater than zero.
LE	Argument is less than or equal to zero.
GE	Argument is greater than or equal to zero.
DF	Argument is completely defined at this point.
NDF	Argument contains one or more undefined symbols at this point.
B	Argument (a string of ASCII characters) is blank or null.
NB	Argument (a string of ASCII characters) is not blank or null.

7.3 SUBCONDITIONALS

There are three subconditional directives that alter the normal conditional processing within a conditional block. These subconditionals (IFF, IFT, IFTF) require no other parameters and must be used within the source code between the IF and ENDC statements. The following functions may be performed through the use of subconditionals:

- Assembly of an alternate block of code when the main conditional code is bypassed because of a failed conditional test.
- Assembly of a noncontiguous body of code within the conditional block depending on the result of the main conditional test.
- Unconditional assembly of a block of code within a conditional block regardless of the results of the conditional test.

The three subconditionals and their functions are:

IFF	The source lines following the IFF statement up to the next subconditional or end of main conditional are assembled if the main conditional test result was false.
IFT	The source lines following the IFT statement up to the next subconditional or end of main conditional are assembled if the main conditional test result was true.
IFTF	The source lines following the IFTF statement up to the next subconditional or end of main conditional are assembled regardless of the main conditional test result.

7.4 NESTING OF CONDITIONALS

Conditionals and subconditionals may be nested to a maximum depth of 64 levels (this includes nesting from macro recursion). Any conditionals within a higher level conditional will be bypassed (the test will not be performed) if the result of the higher level conditional test was false. Subconditionals within outer level conditional blocks will be tested while those within inner level untested blocks will be ignored. Consider the following simple example:

```
TEST1:  IF      EQ,3-3  ; True, so assemble following code
        WORD    33      ; Assembled since EQ,3-3 was true
        IF      NE,4-4      ; False, so bypass following code
        WORD    44          ; Not assembled, since NE,4-4 was false
        IFF                     ; Tested - true since NE,4-4 was false
        WORD    441             ; Assembled since IFF was true
        IFT                     ; Tested - false since NE,4-4 wasn't true
        WORD    442             ; Not assembled since IFT was false
        IFTF                    ; Tested - true regardless of NE,4-4
        WORD    443             ; Assembled since IFTF was true
        ENDC                ; End of NE,4-4 conditional block
        ENDC            ; End of EQ,3-3 conditional block
TEST2:  IF      EQ,5-6  ; False so bypass following code
        WORD    56      ; Not assembled since EQ,5-6 was false
        IF      EQ,6-6      ; Not tested since EQ,5-6 was false
        WORD    61          ; Not assembled since EQ,6-6 was untested
        IFF                     ; Not tested, since EQ,6-6 was untested
        WORD    661             ; Not assembled since IFF was untested
        IFT                     ; Not tested since EQ,6-6 was untested
        WORD    662             ; Not assembled since IFT was untested
        IFTF                    ; Not tested since EQ,6-6 was untested
        WORD    663             ; Not assembled since IFTF was untested
        ENDC                ; End of EQ,6-6 conditional block
        ENDC            ; End of EQ,5-6 conditional block
```

The system macro for the PUSH convenience opcode is a good example of how conditionals may be used to control the code generated by a macro:

```
DEFINE  PUSH    SRC
        IF      B,SRC, CLR -(SP)
        IF      NB,SRC, MOV SRC,-(SP)
        ENDM
```

If the macro above is called without an argument (that is, SRC is blank), then the first conditional is true and the code CLR –(SP) is generated to push a zero word onto the stack. The second conditional code is thus false, and generates no code. If the macro is called with an argument (that is, SRC is not blank), then the reverse happens and the code MOV SRC,–(SP) is generated with SRC being replaced by the real argument in the calling statement. This pushes the SRC word onto the stack.

The same PUSH macro could have been coded in another way using subconditionals:

```
DEFINE  PUSH    SRC
        IF      B,SRC
        CLR     -(SP)
        IFF
        MOV     SRC,-(SP)
        ENDC
        ENDM
```

For more examples of conditionals used within macros, print out or look at the system library file MAC:SYS.M68 which defines all of the supervisor calls used by the AMOS computer system.

Part 2
Using the Alpha Micro Assembly Language Programming System

The chapters that follow describe the use of:

M68	The Macro assembler
LNKLIT	The linkage editor
SYMLIT	The symbol table generator
LIBLIT	The object file library generator
GLOBAL	The global symbol cross reference generator

For information on the screen-oriented, assembly language program debugger, AlphaFIX, see the *AlphaFIX User's Manual*.

CHAPTER 8

THE ALPHA MICRO ASSEMBLER (M68)

This chapter discusses the Alpha Micro macro assembler program, M68, and gives information on using it.

After creating your source code (.M68) file, you must assemble it. The assembler translates your assembly language program into machine language (the .OBJ file). The linkage editor (discussed in the next chapter) processes the .OBJ files to resolve all symbol references and to create the final, executable program (.LIT or .OVR) file.

8.1 THE M68 PHASES

The assembler actually runs in five distinct phases that are selectively called depending on what functions are needed. A brief summary of their respective functions follows:

M68 Phases

PHASE	FUNCTION
Phase 0	Interprets the command line and sets up parameters in the common area for use by successive phases.
Phase 1	Reads the source (.M68) file and performs Pass 1 of a standard two-pass assembly process by expanding macros, building the user symbol table, and generating the interphase work (.IPF) file.
Phase 2	Reads the interphase (.IPF) file and performs Pass 2 of a standard two-pass assembly process by resolving symbols and generating the object code (.OBJ or .UNV) file. M68 then deletes the interphase work file, which may be entirely memory-based.

(Continued)

M68 Phases (Continued)

PHASE	FUNCTION
Phase 3	Reads the source (.M68) file and the object or universal (.OBJ/.UNV) file and creates a list (.LST) disk file or outputs the assembly listing to the terminal. This phase is optional, and is only executed if an error exists in the source files, or if a /L or /R switch is specified or implied.
Phase 4	Actually not part of the assembler but an automatic call to LNKLIT to read the object (.OBJ) file and create a runnable program (.LIT or .OVR) file. Only occurs if there were no internal or external symbol references in the program. (If Phase 4 is not called, you will later have to use LNKLIT to link this file with the other files that contain the symbols that will resolve the external and internal references.)

8.2 COMMAND LINE

The general format for the assembler command line is:

```
M68 filespec{/switch}
```

Filespec identifies the source file you want to assemble—it may optionally be a complete file specification containing account and device specifications. The assembler options selected by ***/switch*** are discussed below.

8.2.1 Assembler Options

An M68 option request (called a *switch*) is a slash followed by one or more letters. A switch alters the normal assembly process. If you enter no switches, M68 performs an assembly on the specified source file and creates an object file but no list file (i.e, Phase 3 is bypassed).

If the program is a single segment (i.e., it contains no INTERN or EXTERN statements), then M68 enters Phase 4, which creates an executable (.LIT or .OVR) program file.

You may select one or more of the assembly options below by specifying the appropriate switch on the M68 command line:

M68 Option Switches

OPTION	FUNCTION
/B text	Generates a bottom footer line on every page of the listing using the rest of the text on the command line following the /B switch as title information. For example: `M68 DEVCPY/B Version A00` RETURN generates a listing file of which every page contains the bottom line title: "Version A00." /B must be the last switch on the command line.
/C	Includes conditionals in the listing. Conditionals are normally suppressed.
/E	Writes to the assembly listing only those lines that contain an error.
/L	Generates a list file by calling Phase 3 during the assembly. The output file has the same name as your source file and a .LST extension. You may modify the list file name by using the OBJNAM pseudo opcode in your source program. See Chapter 5 for information on OBJNAM.
/M20	Allows the assembler to accept 68020 instructions and addresssing modes. (A syntax error S will be generated on assembly if your program contains such instructions or addressing modes, and you do not use the /M20 switch.) For information on the 68020 extensions, see the latest version of the *Alpha Micro Instruction Set Manual*.
/M40	Allows the assembler to accept 68040 instructions and addresssing modes. (A syntax error S will be generated on assembly if your program contains such instructions or addressing modes, and you do not use the /M40 switch.) For information on the 68040 extensions, see the latest version of the *Alpha Micro Instruction Set Manual*.
/N	Does not force the assembler to fold all lower case letters to upper case letters. If this switch is not used, the assembler defaults to folding lower case letters to upper case letters.
/O	Uses the current object file by omitting Phases 1 and 2.
/R	Generates a cross reference at the end of the assembly listing. See later in this chapter for information on the cross reference listing.

(Continued)

M68 Option Switches (Continued)

OPTION	FUNCTION
/S	Normally, during Phase 2, M68 displays all EXTERNed symbols. If you use the /S switch, these symbols are not displayed.
/T	Displays the assembly listing on your terminal instead of writing it to a disk file.
/V{a}:x	Allows you to specify a value on the M68 command line which can be examined during the assembly process. ***a*** specifies the type of value specified, and ***x*** is the value. See the next section for more information.
/X	Lists all macro expansions in your assembly listing. M68 expansions are normally suppressed.

You do not have to specify the /L switch when you use the /B, /C, /E, /R, /T, or /X switches to tell M68 to generate a listing.

8.3 USING M68

The most common method of assembling new programs is as follows:

1. Assemble the program with the command:

   `M68 filespec` RETURN

   This allows you to count any errors that occur during Phases 1 and 2.

2. If no errors occur, create a list file with:

   `M68 filespec/LO` RETURN

   or, optionally, list it on the terminal with:

   `M68 filespec/TO` RETURN

   or, get a cross reference with the listing:

   `M68 filespec/RO` RETURN

3. If there were errors, list them alone with:

```
M68 filespec/TOE RETURN
```

Correct the errors and go back to Step 1.

If the program has only one segment, M68 automatically calls Phase 4 which creates the .LIT or .OVR program file—otherwise, you need to use LNKLIT or SYMLIT to generate the final program file—see the next chapter for information on LNKLIT and SYMLIT. /els

8.3.1 Parameterized Assembly Option

M68 provides a parameterized assembly option—you can use the /V switch to specify a value on the M68 command line. The NVALU pseudo opcode allows your program to access the value specified in the /V assembly switch. The NVALU statement takes the form:

```
NVALU sym
```

which sets the symbol ***sym*** to one of the values shown in the table below, depending on which form of the /V switch is used. To use the /V switch, place it at the end of your M68 command line. For example:

```
M68 MYPRG/VD:10 RETURN
```

For each form of the /V switch, the table below shows what type of data may be included with the switch, and what the switch sets the NVALU symbol to.

SWITCH	:X	NVALU SYMBOL
/V:x	x is an octal or hex number (depending on the current radix being used by the assembler).	symbol is set to x.
/VO:x	x is an octal number.	Symbol is set to ^0x.
/VH:x	x is a hexadecimal number.	Symbol is set to ^H0x.
/VD:x	x is a decimal number.	Symbol is set to ^Dx.
/VA:x	x is one or two ASCII characters.	Symbol is set to ’x if a single ASCII character, "x if two ASCII characters.
/VR:x	x is one to three RAD50 characters.	Symbol is set to [x].

You may find this feature especially useful when using conditional assembly directive pseudo opcodes to select which portions of code to assemble.

8.4 SAMPLE ASSEMBLY DISPLAY

Below we show a sample assembly display:

```
M68 SAVTXT.M68/L RETURN

== AM-100/L Macro Assembler Version X.X(XXX) ==

Processing SAVTXT.M68

Phase 1:   Searching DSK0:SYS.UNV[7,7]
           Work area: 3916 bytes, 3614 used
Phase 2:   Object file finished
Phase 3:   Listing file finished
Phase 4:   Program file finished
           [Program size = 60. bytes]
```

If M68 automatically EXTERNs any symbols, it lists them alphabetically in Phase 2 (unless you use the /S switch). For example:

```
Phase 2:   Object file finished
EXTERNs were generated for the following symbols:
     GETNUM  PRTNUM
```

In the case above, M68 automatically EXTERNed the symbols GETNUM and PRTNUM. M68 automatically EXTERNs symbols if those symbols are undefined and if the AUTOEXTERN pseudo opcode appears in your source file.

Even if your program is a single segment, M68 does not call Phase 4 to link your program if it is not able to resolve all symbol references in your program (that is, if EXTERNs were generated). You will need to use LNKLIT or SYMLIT to link your program with other file(s) that contain the symbols referenced by your main program.

If you ask for a cross reference listing, you see the following message during Phase 3:

```
Phase 3:   Cross reference file finished
```

8.5 THE ASSEMBLY LISTING

By specifying the appropriate assembly switches, you can direct M68 to call Phase 3 of the assembly process to create a list file which is sent to a disk file or to your terminal. The listing is formatted and contains both the source of your program and numeric code that is generated by the assembly.

8.5.1 Assembly Listing Format

Each page contains a page number and a title giving the name of the program that has been assembled and the account number the file was assembled in. Unless otherwise controlled by PAGE statements, each page contains 54 lines of source data. Each page is terminated by a form-feed character. The system date appears at the top of each page of the listing. If you specified the /B assembly switch, M68 outputs to each page a page footer containing the text specified on your M68 command line.

Each data line on the listing contains four sections:

1. Columns 1-5 list the error codes on the line that generated the error. For a list of the M68 error codes, see "M68 Errors," below.

2. Columns 8-13 list the current address of the generated data if any data code was generated. Or, these columns give the value of the assignment if this is an equate statement.

3. Columns 16-37 list the genrated binary data (maximum of the first six words) in hex.

4. Columns 40-132 list the source line.

8.5.2 Listing Control Pseudo Opcodes

Several pseudo opcodes exist to control your assembly listing—you will place these pseudo opcodes in your source program. We list them briefly below. For more information on each pseudo opcode, see Chapter 5.

OPCODE	FUNCTION
LIST	Re-enables output to the listing file.
NOLIST	Turns off output to the assembly listing file. LIST and NOLIST are ignored if you use the /X switch.
PAGE	Begins a new page in the assembly listing.

8.5.3 Generating a Cross Reference

You may use the /R switch to generate a cross reference as part of the assembly listing. To see the cross reference on your terminal, use the /RT switches. You may specify the /O switch to bypass assembly Phases 1 and 2 if an object (.OBJ) file for the current source field aready exists.

For information on using the GLOBAL command to generate a global cross reference, see Chapter 11.

Cross Reference Control Pseudo Opcodes

The CREF, MAYCREF, and NOCREF, pseudo opcodes control the generation of the cross reference listing:

OPCODE	FUNCTION
CREF	Enables normal cross referencing.
NOCREF	Suppresses from the listing all defined symbols until M68 encounters a CREF or MAYCREF pseudo opcode.
MAYCREF	Suppresses from the listing all defined symbols if those symbols are never referenced.

Cross Reference Listing Format

The cross reference listing is similar to an ordinary assembly listing except that it also includes the following:

- A column of sequence numbers appears at the left of the listing.
- At the end of the assembly listing, an alphabetic listing of each symbol appears giving, in numeric order, the sequence numbers of the lines in which each symbol appears.

  These sequence numbers are sometimes followed by a code of the form –X where X identifies the type of symbol. X may be one of the following:

L A label definition
E An equate definition
I An INTERNed symbol
X An EXTERNed symbol
O An OVRLAY.

Also, a single quote (') appears after symbols that were never defined. M68 automatically EXTERNs such symbols if the AUTOEXTERN pseudo opcode is present in your source program.

- A similar listing of macro definitions and references follows the symbol listing. The sequence number corresponding to a macro definition is flagged by a –M code.

Sample Cross Reference Listing

The cross reference appears at the end of a regular assembly listing. Below is a sample of what the cross reference portion of the assembly listing for a small program, MATH.M68, might look like:

```
MATH    [110,5]             Cross Reference Listing          1-May-86
ACCUM   394    434    520    530    542    543    553    554
ADD     423    520-L
DIVI    429    553-L
EXIT    365    370    459    597-L
GETEXP  364-L
GETNUM' 386    415
NUMERR  393    416    567-L
OPRERR  407    411    583-L
OPRTBL  468    613-L
PARSE   383-L
PRTNUM' 441    450
START   354-L
SUB     425    530-L
S..RDX  30-E   309
$VAL    615    616-E  618    619    619-E  623    626    627
        643    643-E

MATH    [110,5]             Cross Reference Listing          1-May-86
BYP     181-M  366    383    398    412    494    503
CRLF    173-M  457
EXIT    174-M  573    589    597
GTDEC   186-M  496
OPERAT  331-M  613    621    629    637
TYPECR  292-M  451    567    583
```

The cross reference above identifies equated symbols and macro definition symbols. It also identifies the GETNUM and PRTNUM symbols as undefined or automatically EXTERNed symbols.

8.6 M68 ERRORS

M68 displays two types of error messages—error codes that appear in your assembly listing and error messages that appear on your terminal screen as you assemble the program.

8.6.1 Error Codes

Below are the error codes that can appear in your assembly listing. Each code appears on the line of the source program in which the error occurred.

M68 Error Codes

CODE	ERROR
A	Branch address was out of the 127-word range.
B	Boundary error—a word operand was on an odd byte address. See Chapter 5 for information on the EVEN pseudo opcode.
C	Conditional statement syntax error.
D	Duplicate user symbol. (Symbol defined more than once.)
E	User-generated error through use of ASMERP.
I	Illegal character in source line.
M	Missing term or operator in operand or expression.
N	Numeric error which indicates a digit out of the current radix range.
O	The specified operand is illegal when used with this instruction.
P	An expression that had to be resolvable on the first pass was not.
Q	Questionable syntax—this is a general catch-all error code.
R	Register error—a register expression was not in the range 0-7.

(Continued)

M68 Error Codes (Continued)

CODE	ERROR
S	Indicates an offset requiring greater than 16 bits, which is not allowed unless using 32-bit specific addressing forms or M68 /M20 or /M40 switch. (See "M68 Option Switches" for information on /M20 and /M40.)
T	Source line or operand terminated improperly.
U	Undefined user symbol during Pass 2.
V	Value of an absolute parameter was out of its defined range.
X	Assembler system error—please notify Alpha Micro.

8.6.2 Error Messages

You may see several error messages during the program assembly:

%Assembly Fold - No Fold switch doesn't match .UNV file

This can be caused by two circumstances. The first is if you used the /N switch when the .UNV file was being created and you are not using it on the current assembly. The second is if you did not use the /N switch when the .UNV file was created and you are using it on the current assembly. In either case, symbols may be undefined since the assembler was folding lower case letters to upper case in the instances where the /N switch was not used.

?Cannot OPEN Devn: - invalid filename

There is something wrong with the format of your command line. For example, you may have tried to use an assembly switch but forgot to place it at the end of the file specification. All switches must appear at the end of the command line.

Copy file filespec not found

filespec is the file specification you supplied to the COPY pseudo opcode. For detailed information on the search pattern M68 uses to search for the copy file, see Chapter 5.

?Expression stack error

This is an internal M68 error. You should never see it—but if you do, check your source program to see if you made any errors in specifying expressions.

?File specification error

There is something wrong with the format of your command line. For example, you typed M68 and pressed RETURN, omitting the file specification.

?Invalid .UNV file version number - aborting

You must recreate your .UNV file by reassembling the source program using the current version of M68.

?M68n.OVR not found

where n is a number identifying the particular M68 overlay file. M68 cannot find one of the overlays that are a part of M68. Make sure that the file is in account DSK0:[1,4]. If the file is not there, contact the System Operator.

%Old format .UNV file - assembly being done with No fold option

The .UNV file was created using the old version of M68 which did not allow symbols longer than six characters or lower case letters in symbols. The current assembly is being done with the /N switch. This may cause problems with symbol definitions in the program being assembled since M68 will not fold lower case symbols to upper case and does not truncate symbols to six characters.

?Search file filespec not found

where filespec is the file specification you supplied to the SEARCH pseudo opcode. For information on SEARCH, see Chapter 5.

[SYNC ERROR]

M68 generates a listing file by reading the source file and the object file and synchronizing the source lines with the resolved object data to come up with the listing line data. If these two files get out of sync, there is no way the listing may proceed and the message [SYNC ERROR] appears on your terminal. M68 will then close the list file at the point of the sync error, but the line that caused the error will not have been included. A sync error of this sort means one of two things—either you have an out-of-date object file that you are using with the /O switch, or you have found an undiagnosed assembler bug. These bugs usually occur when you get fancy with nested macros and conditionals that have a valid error buried down deep within.

CHAPTER 9

LINKAGE EDITOR (LNKLIT) AND SYMBOL TABLE FILE GENERATOR (SYMLIT)

This chapter contains information on the linkage editor LNKLIT and the symbol table generator SYMLIT. We discuss both of these programs at this time because LNKLIT and SYMLIT are very similar and, with the proper selection of option switches, can be made to perform virtually the same functions. LNKLIT takes one or more object files produced by the assembler and resolves all external symbol references. The file LNKLIT produces is the final, executable program file. SYMLIT takes one or more object files and produces a symbol table file for that program. As we will see later, LNKLIT and SYMLIT can also perform other functions.

Besides discussing how to link .OBJ files, this chapter discusses the use of LNKLIT and SYMLIT with library (.LIB) files. For more information on object file libraries, see Chapter 10.

9.1 LNKLIT

The assembler itself does not produce a file that is usable as an executable program, unless your program is a single segment file that contains no EXTERNed, INTERNed, or AUTOEXTERNed symbols, in which case M68 calls LNKLIT as Phase 4 of the assembly.

The assembler output file is an object (.OBJ) file that is not fully resolved and which contains symbol definitions and embedded cross-segment commands.

The linkage editor (LNKLIT) resolves the object file. LNKLIT reads one or more of these object files and creates one runnable program (.LIT) file which the operating system can load into memory and run. Furthermore, if the program contains overlay segments, LNKLIT resolves them and creates one overlay (.OVR) file for each one. These overlay files are loaded into memory upon command during the running of the program and allow memory conservation for large programs such as the assembler itself.

9.1.1 LNKLIT Command Line

The general format of the LNKLIT command is:

```
LNKLIT {/switches}filespec1{filespec2,..filespecN}{/switches}
```

where ***filespec*** selects an object file. The default extension is .OBJ. The first file specified cannot be a library file or an overlay file. If a filespec includes a device and account specification, LNKLIT searches for the file in that account. If you omit a device and account specification, LNKLIT searches for the file first in the account and device you are logged into; secondly, in your project library account (account [P,0]); and, finally, in the System M68 Library account, DSK0:[7,7].

Continuation Lines

If the program you want to link contains more files than will fit on the command line, you may continue the files on the next line by ending the last filespec with a comma. LNKLIT then displays an asterisk and waits for more filespecs. LNKLIT continues to accept files as long as the last filespec on the line terminates with a comma.

LNKLIT Options

LNKLIT treats switches in the same way that a standard AMOS wildcard command does—this means that the files affected by the option switches you use can depend on where you place the switches. Any switch in front of a filespec becomes the default switch and thus affects the rest of the filespecs on the command line (unless canceled by a subsequent switch). Any switch at the end of a filespec affects only the files selected by that specification. For example, suppose you want to use the /O switch to identify one or more .OBJ files as optional files:

```
LNKLIT FILBCK,/O DIRBCK,TAPBCK RETURN
```

selects the files DIRBCK and TAPBCK as optional files because the /O switch **precedes** the filespec DIRBCK, and thus becomes the default. The command line:

```
LNKLIT FILBCK,DIRBCK/O,TAPBCK RETURN
```

selects only the file DIRBCK as an optional file because the /O switch follows the DIRBCK filespec and appears before the next comma in the command line.

Special switches (identified as "operation switches" in the discussions below) affect ALL filespecs specified on the command line no matter where you place the switch. For example, it does not matter where you place the /M switch on the command line—it affects all files selected by the filespecs on the command line.

LNKLIT Option Switches

OPTION	FUNCTION
/B	Don't force word alignment of .OBJ modules.
/D	Generates a Run Time Initializer (.RTI) file. See "The Run Time Initializer File," in this chapter for information on the .RTI file. *Operation switch.*
/E	Includes equated symbols in the symbol table file. You must use /E with the /M or /S switch. *Operation switch.*
/L	Designates a library file. See "Library and Optional Files" in this chapter for information on library files.
/M	Generates a load map (.MAP) file. See "The Load Map File" in this chapter for a discussion of the load map. *Operation switch.*
/N	Suppresses /P switch. *Operation switch.*
/O	Designates an optional file. See "Library and Optional Files" in this chapter for information on optional files.
/P	Generates program (.LIT) and overlay (.OVR) files. The default switch. *Operation switch.*
/R	Designates a required file. The default switch. Cancels the /L and /O switches.
/S	Generates a symbol table (.SYM) file. *Operation switch.*
/U	Don't automatically include SYSLIB.LIB in linkage.
/X	Allows a maximum DSECT area of 64Kb rather than the default 32Kb value by allowing use of full, 16-bit signed offsets. Initializes DSECT location counter to –32768. Refer to Chapter 5 for information on DSIZE value. *Operation switch.*

You may specify multiple switches by preceding each switch with a /, as in the sample command line below.

9.1.2 Sample LNKLIT Display

Below is a sample LNKLIT display. The sample below uses the /L switch to specify a library file and the /M switch to generate a load map.

```
LNKLIT MATH,UTILIT.LIB/L/M RETURN

== AM-100/L Linkage Editor Version X.X(xxx) ==

Processing MATH.OBJ [Base = 0, Size = 348. bytes]

-- Optional and Library Request --

Processing UTILIT.LIB(NUM) [Base = 534, Size = 144. bytes]

Program and Map files finished.
[Program size = 492. bytes]
```

LNKLIT tells you the size (in decimal bytes) of each module. If you specify a library file, LNKLIT tells you which of the object files in the library file are being linked in. In the sample above, LNKLIT linked in the NUM routine from the UTILIT.LIB library file.

If a data section was defined via the DSECT pseudo opcode, and its size is greater than zero, LNKLIT tells you the size of the data section.

9.1.3 LNKLIT Errors

LNKLIT reads each of the files specified and creates the necessary program and optional overlay files. LNKLIT displays any error messages on the terminal if it encounters any errors during processing. The most common error is the undefined global symbol error which means you have an EXTERN symbol in one segment which is not defined in another segment by an INTERN statement. LNKLIT does not generate a program file if it cannot find one or more of the segments in its assembled object (.OBJ) form. For a list of LNKLIT error messages, see the end of this chapter.

9.2 THE SYMBOL TABLE FILE GENERATOR (SYMLIT)

The object files output by the assembler contain complete information on the symbols used in your program, as well as the actual generated code. To make this list of symbols available to the debugger program, AlphaFIX, you use SYMLIT. Like LNKLIT, SYMLIT takes one or more .OBJ files and creates an output file—in this case, a symbol (.SYM) file. AlphaFIX uses this file for symbolic program debugging.

Unlike the program file, the symbol file is not generated automatically even if only one program segment is used. You must use SYMLIT if you wish to create a symbol file.

9.2.1 SYMLIT Command Line

The format for calling SYMLIT is identical to the LNKLIT command line:

```
SYMLIT {/switches}filespec1{,filespec2,...filespecN}{/switches}
```

where ***filespec*** selects an object file. The default extension is .OBJ. The first file specified cannot be a library file or an overlay file. If a filespec includes a device and account specification, SYMLIT searches for the file in that account. If you omit a device and account specification, SYMLIT searches for the file first in the account and device you are logged into; secondly, in your project library account (account [P,0]); and, finally, in the System M68 library account, DSK0:[7,7].

The output of SYMLIT is placed into a file named filespec.SYM, where filespec is the first file specified on the SYMLIT command line. No symbol file will be generated if one or more of the specified files is not found in its assembled object (.OBJ file) form.

Continuation Lines

As with LNKLIT, if the program contains more files than will fit on the command line, you may continue the file specifications on the next line by ending the last filespec with a comma. SYMLIT then displays an asterisk and waits for more filespecs. SYMLIT will continue to accept filespecs as long as the last filespec on the line ends with a comma.

SYMLIT Options

SYMLIT handles option requests in the same way as LNKLIT. (See the previous section on LNKLIT for information on how the placement of switches on the command line influences which files are affected by the option request.)

SYMLIT Option Switches

OPTION	FUNCTION
/B	Don't force word alignment of .OBJ modules.
/D	Generates a Run Time Initializer (.RTI) file. See "The Run Time Initializer File" in this chapter for information on the .RTI file. *Operation switch.*
/E	Includes equated symbols in the symbol table file. You must also use this switch with /M to tell SYMLIT to include equated symbols in the load map. *Operation switch.*
/L	Designates a library file. See "Libary and Optional Files" in this chapter for information on library files.
/M	Generates a load map (.MAP) file. See "The Load Map File" in this chapter for a discussion of the load map. *Operation switch.*
/N	Suppresses /P switch. *Operation switch.*
/O	Designates an optional file. See "Library and Optional Files" in this chapter for information on optional files.
/P	Generates program (.LIT) and overlay (.OVR) files. The default switch. *Operation switch.*
/R	Designates a required file. The default switch. Cancels the effect of a /L or /O switch.
/S	Generates a symbol table (.SYM) file. *Operation switch.*
/U	Don't automatically include SYSLIB.LIB in linkage.
/X	Allows a maximum DSECT area of 64Kb rather than the default 32Kb value by allowing use of full, 16-bit signed offsets. Initializes DSECT location counter to –32768. Refer to Chapter 5 for information on DSIZE value. *Operation switch.*

You may specify multiple switches by preceding each with a /. (See the sample command line below.)

9.2.2 Sample SYMLIT Display

Below is a sample SYMLIT display. The command line uses the /L switch to specify a library file, and the /M switch to generate a load map.

```
SYMLIT MATH,UTILIT.LIB/L/M RETURN

== AM-100/L Linkage Editor Version X.X(xxx) ==

Processing MATH.OBJ

-- Optional and Library Request --

Processing UTILIT.LIB(NUM)

Symbol and Map files finished.
```

If you specify a library file, SYMLIT tells you which of the object files in the library file it is including in the symbol table file. In the sample above, SYMLIT included the NUM routine from the UTILIT.LIB library file.

If your program has generated a data section with a size greater than zero, SYMLIT also tells you how large the data section is.

If you compare this display with that of LNKLIT shown earlier, you will notice that it is very similar. In fact, LNKLIT and SYMLIT can be made to perform exactly the same functions. If we had specified the /P switch and the /N switch (specifying that we wanted a .LIT file generated and did not want a symbol table file), the display above would have looked exactly like the LNKLIT display shown previously.

9.3 LIBRARY AND OPTIONAL FILES

Both LNKLIT and SYMLIT support the use of library files and optional files.

Most programmers have been faced at one time or another with the task of writing a standard routine again and again for multiple programs. Library and optional files help you avoid this situation by allowing your programs to contain references to previously written routines in an object file library or an optional file.

Besides making life easier by making it possible for you to write frequently used routines only once, library and optional files also help to standardize programs by providing the same error checking, input checking, message display, etc., for multiple programs.

LNKLIT and SYMLIT place any object files from a library file and any optional files at the end of your program in the order they are needed to resolve external references.

9.3.1 Library Files

A library file is a file produced by LIBLIT (discussed in the next chapter). The library file contains a group of .OBJ files. The purpose of generating a library file is to gather together a group of subroutines used frequently by programs on your computer. These routines are then easily accessed by all programmers on the system by using the EXTERN or AUTOEXTERN pseudo opcodes in their source programs and specifying the required routine. Unlike using the COPY pseudo opcode, which physically incorporates the entire source file specified into your assembled program when you assemble it, using a library file causes only those library subroutines that are referenced by your program to be linked into your program.

The entire .OBJ file that is a component of a library file will be linked in if your program references a symbol in it—not just that portion of the .OBJ file required by your program. For example, suppose you create a library file (using LIBLIT) that contains the following .OBJ files: STRCHK, GETLIN, and GETNUM. If your program references a symbol within the GETNUM object file, the entire GETNUM file is linked in even if it also contains several other routines. For this reason, you should limit each .OBJ file that is a component of the library to one subroutine.

You may not specify the library file first on the LNKLIT or SYMLIT command line. To resolve symbol references, LNKLIT and SYMLIT must first access the file that makes those references before it accesses the file that defines them.

9.3.2 Optional Files

By using the /O switch with LNKLIT or SYMLIT, you may request the specified file (called an "optional file") be included in the linked program if the optional file is needed to resolve any external references in one of the other files being linked. If such a reference exists, the optional file will be incorporated into your program—otherwise, it will not. Unlike a library file, an optional file only contains the contents of a single .OBJ file. An optional file may not be an overlay.

9.4 THE LOAD MAP FILE

A load map shows how the modules linked together will be loaded into memory when the program is invoked for execution. Using the /M switch with LNKLIT or SYMLIT, you may ask that a load map file be generated. A load map file has the name of the first file specified on the LNKLIT or SYMLIT command line and the extension .MAP.

A load map lists each object file used in the order that it was used. The base and size of each object file's data section is shown. For each object file, the load map gives the following information:

1. The file's offset from the beginning of the program;

2. The size of the file in decimal bytes;

3. In alphabetical order, all the symbols defined in that file and their relocated values after the linking process. If the symbols are relocatable relative to the base of the program, the load map flags them with a "r" symbol. If the symbols are relocatable relative to the base of the data section, the load map flags them with a "d" symbol.

For example, the following LNKLIT command line:

LNKLIT MATH,NUM/M RETURN

generated the load map file below, MATH.MAP:

```
[AM-100/L Linkage Editor Version X.X(xxx)]
Program Load Map
================
        PSECT                        DSECT
Module    Base    Size         Base       Size
------    ------------         ----       ----
   Symbol                           Value
   ------                           -----

MATH   00000000000r  2122.    00000000000d   0.

   ABORT                            00000003224r
   DEVBYT                           00000003076r
   DEVCHG                           00000002566r
   DEVCLR                           00000003016r
   DEVNEW                           00000002712r
   DEVNON                           00000003049r
   GETB.D                           00000000664r
   GETDEV                           00000003030r
   IE.BAD                           00000004052r

NUM    00000004112r  4280.    00000000000d   0.

   DCLOSE                           00000006470r
   FISHX                            00000007744r
   FLSH.1                           00000007624r
   FLUSH$                           00000007560r
   ICL.OO                           00000006606r
   ICL.EO                           00000006462r
```

9.5 THE RUN TIME INITIALIZER FILE

The Run Time Initializer (.RTI) file is used by the Run Time Initializer (RTI) program at execution time. The RTI program is invoked by the GETRTI macro call in your program (see the manual *AMOS Monitor Calls* for more information on GETRTI). The .RTI file is generated by LNKLIT when the /D switch is specified. The .RTI file contains information that is used at run time to initialize the impure zone. Refer to Chapter 4 for more information on DSECT initialization.

If the /D and /X switches are both specified, then the value returned by the DSIZE parameter is a two's complement number. You must be certain to negate this value before using it in the GETRTI macro call, because the GETRTI macro is expecting a positive value. In addition, if you insert instructions in the DSECT and use both the /D

and /X switches, then the assembly and linkage may cause negative offsets to be generated in the DSECT. This is because the /X switch causes the DSECT to start at –32K, so DSECT offsets may be 32K smaller than the value you are expecting. The Run Time Initializer is unable to compensate for this smaller value and the code may not be properly executed or may cause unpredictable results.

9.6 LNKLIT AND SYMLIT ERROR MESSAGES

You may see the following error messages when using LNKLIT and SYMLIT:

Attempt to directly reference a DSECT symbol

You must always index through a base register when referencing a symbol defined in a data section. For more information on DSECT and data sections, refer to Chapter 4.

?BYTE size exceeded (xxxx) at location nnnn

The BYTE argument was less than -255 or greater than 255. xxxx is the name of the label, and nnnn is the bute offset into the current module where the error occured.

?Command error

There was something wrong with your command line. For example, you tried to use LNKLIT or SYMLIT without specifying a file on which to work.

?Fatal error—Attempt to specify overlay xxx as optional

You may not use the /O switch to designate a file as optional if that object file is an overlay.

?Fatal error—DSIZE must not appear in an overlay
?Fatal error—At most one DSIZE may appear in a program

You are using the DSIZE pseudo opcode incorrectly. For information on using the DSIZE pseudo opcode to define the size of a data section, see the section on DSIZE in Chapter 5.

?Fatal error—Expression stack error

An error occurred when LNKLIT or SYMLIT evaluated some expressions in your files. This indicates an internal error—you should never see this error message.

?Fatal error—Expression stack overflow

You exceeded the number of nested expressions that LNKLIT or SYMLIT can handle. Try to find the exceedingly complex expression in your source file and simplify it.

?Fatal error—First file must not be a library

To enable LNKLIT or SYMLIT to correctly resolve external references to a library, you must specify the program that references that library before you specify the library file itself.

?Fatal error—First file must not be an overlay

To enable LNKLIT or SYMLIT to correctly resolve external references to an overlay, you must specify the program that references that overlay before you specify the overlay file itself.

?Fatal error—Incompatible library file version
?Fatal error—Incompatible object file version

You are using an obsolete object file. Make sure you have not inadvertently mixed object files from a previous version of the operating system with your current assembler software. Make sure the SYSLIB.LIB file you are using is the correct version for your assembly language system.

?Fatal error—Insufficient memory

You must increase the size of your memory partition—there was not enough room to perform the procedure you specified.

?Fatal error—Overlays of code are not permitted Next expected address is xxxx Overlay code address is xxxx

Your program is trying to overlay previous code. Check your .M68 programs to make sure that your overlay references are correct.

?Fatal error—Overlay symbol "xxxx" in segment yyyy
was not defined in a previous input segment

You may not reference an undefined overlay. In other words, LNKLIT is trying to process a supposed overlay file, but has seen no references to the overlay in a previous file. Without such a reference, LNKLIT cannot construct the overlay, so it aborts and returns you to AMOS command level.

?Fatal error—Overwriting of impure zone not permitted
Next expected address is xxxx
Overwriting address is xxxx

You have backstepped the DSECT location counter, thus overwriting previousl allocated storage within a DSECT area.

?Undefined switch /x—ignored

Refer to Appendix A, "Summary of Program Switches," to make sure you specified a valid switch.

?WORD size exceeded (xxxx) at location nnnn

The WORD argument was out of range. xxxx is the name of the label, and nnnn is the bute offset into the current module where the error occured.

?xxxx undefined

An external symbol is undefined. This is a very common error. You referenced a symbol which has not previously been defined (e.g., you made a reference to a label that does not exist). Make sure an EXTERN statement in one segment is defined by an INTERN statement in another segment.

CHAPTER 10

THE OBJECT FILE LIBRARY GENERATOR (LIBLIT)

One of the more aggravating programming tasks is rewriting a utility program you've used many times before and you know you will use many times in the future. Many kinds of routines are so useful that you need them again and again in many different programs: e.g., routines that check for ASCII characters, input and output characters, sort data, etc.

The purpose of the library file is to collect together these frequently used routines where they can be accesible to your program files when you link them into final, executable programs. Not only do library files help you to avoid writing and rewriting the same routines over and over, but they can also give help to every other programmer on the computer. An added benefit of library files is that they tend to help standardize programs by providing standard input, output, error checking, and message display routines used by everyone on the computer.

The Alpha Micro object file library generator, LIBLIT, constructs library files out of .OBJ files. Each of the .OBJ files built into the library file is a separate routine that can be accessed by your programs. The final library file has a .LIB extension and can be used by both LNKLIT and SYMLIT.

10.1 LIBLIT COMMAND LINE

The LIBLIT command line takes the form:

```
LIBLIT{/L} output=input1{,input2,...inputN}
```

(A shorthand version is discussed later.)

Output specifies the name of the library file you want to create. The output file has the extension .LIB and the name specified.

Input specifies the .OBJ files you want to place in the library. The input specification can take the following forms:

filespec
filespec\item1
filespec\(item1,item2,...itemN)
filespec(item2,item2,...itemN)

The \ symbol designates an exception. For example, the command line:

```
LIBLIT MYLIB\SUB1,NEWSUB,READIT RETURN
```

tells LIBLIT that you want to modify the existing library MYLIB (the "inout" specification) by removing the object file SUB1, and adding NEWSUB and READIT.

The parentheses specify a group of object files. For example:

```
LIBLIT MYSLIB\(SUB1,NEWSUB,READIT),GETNUM RETURN
```

tells LIBLIT to modify the existing library MYLIB by deleting the collection of object files SUB1, NEWSUB, and READIT, and to add the object file GETNUM.

LIBLIT looks for the specified files in the account and device specified. If you omit the device and account specification from the filespec, LIBLIT searches first in the account and device you are logged into; then your projet library account on the device you are logged into (account [P,0]); finally, LIBLIT searches in the System M68 library account, DSK0:[7,7].

A shorthand version of the LIBLIT command line exists that allows you to specify a single file as both the input and output file. This is used to update an existing library. The form is:

```
LIBLIT{/L} inout{,input2,...inputN}
```

This is equivalent to:

```
LIBLIT inout=inout{,input2,...inputN}
```

if you do not use the /L switch—otherwise, it is equivalent to:

```
TRM:=inout{,input2...inputN}
```

10.1.1 Continuation Lines

As with LNKLIT and SYMLIT, you may enter as many filespecs as you wish on as many lines as you wish as long as you end the last filespec on the line with a comma. LIBLIT will display an asterisk and wait for more filespecs. LIBLIT will continue to accept filespecs as long as the last filespec on the line ends with a comma.

10.1.2 LIBLIT Option Switch (/L)

The only LIBLIT switch is /L, which tells LIBLIT to generate a library listing. This listing looks similar to a load map listing (see Chapter 9 for a sample), and lists all object files in the library file and all INTERNed symbols.

If you specify an output file (e.g., LIBLIT LIST=MYLIB/L) LIBLIT creates the listing with that name and extension. (The default extension is .LST.) If you do not specify an output file (e.g., LIBLIT MYLIB/L), LIBLIT sends the library listing to your terminal display.

10.2 SAMPLE LIBLIT DISPLAY

Suppose you are creating a new library called USEFUL from the .OBJ files ERRMSG, GETLIN, and FORMAT:

```
LIBLIT USEFUL=GETLIN,FORMAT,ERRMSG RETURN

== AM-100/L Object File Librarian Version X.X(xxx) ==

Processing GETLIN.OBJ
Processing FORMAT.OBJ
Processing ERRMSG.OBJ

Library file finished
```

As LIBLIT processes each new .OBJ file, it tells you so.

Suppose you want to add a routine to an existing library. The sample display might look like this:

```
LIBLIT USEFUL,LINSIZ RETURN

==AM-100/L Object File Librarian Version X.X(xxx) ==

Processing USEFUL.LIB(GETLIN)
Processing USEFUL.LIB(FORMAT)
Processing USEFUL.LIB(ERRMSG)
Processing LINSIZ.OBJ

Library file finished
```

You've successfully added the new routine LINSIZ to the old library that already contained the object files GETLIN, FORMAT, and ERRMSG. LIBLIT tells you as it processes each .OBJ file contained within the library file.

10.3 UPDATING A LIBRARY

Replacing one or more of the .OBJ files that make up a library file can be a bit tricky. If you simply try to add a new version of an existing .OBJ file without deleting the old one first, problems can result because both versions of the object file will be in the library. The recommended procedure is to first delete the old routine, then add the new one. For example, to replace the old version of FORMAT with a new one, you enter:

```
LIBLIT USEFUL\FORMAT, FORMAT RETURN
```

which first deletes the file and then adds it. Assume that our small library only contains three routines, GETLIN, ERRMSG, and FORMAT. The LIBLIT display in response to this command line would look like this:

```
==AM-100/L Object File Librarian Version X.X(xxx) =

Processing USEFUL.LIB(GETLIN)
Processing USEFUL.LIB(ERRMSG)
Processing FORMAT.OBJ

Library file finished
```

LIBLIT tells you what routines are contained in the library.

10.4 LIBLIT ERROR MESSAGES

You may see the following error messages when you use LIBLIT:

?Command error

LIBLIT did not understand your command line. For example, you entered LIBLIT and then pressed RETURN. Make sure your file specifications are in standard form.

?Fatal error - Incompatible library file version
?Fatal error - Incompatible object file version

You are using obsolete library or object files. Make sure you have not inadvertently mixed files from a previous version of the operating system with your current assembler software. Make sure the SYSLIB.LIB file you are using is the correct version for your assembly language system.

?Fatal error - xx is an overlay

You may not specify an overlay as an element of an object file library.

?Listing aborted

LIBLIT was not able to finish the library listing. For example, an error occurred while LIBLIT was trying to access a file.

?Obj files are not libraries -- they cannot be restricted with a modifier

You may only use the \ file restrictor and the () file inclusion symbols if you are modifying a library.

?The following module was not found - xxx

You tried to modify an existing library, but the object files you specified were not present in the library file. Make sure that you did not accidentally use the \ restrictor symbol.

?Undefined switch /X - ignored

where X is the switch you supplied. LIBLIT currently uses only one option switch, /L, to produce a library listing. Make sure you did not type a / by accident when you wanted to type a backslash.

CHAPTER 11

THE GLOBAL CROSS REFERENCE GENERATOR (GLOBAL)

The GLOBAL program takes a group of object (.OBJ) files and produces an alphabetic global cross reference which lists all global symbols in the files, and shows which files define those symbols and which files accept them as externally defined symbols.

In other words, GLOBAL produces a listing file that contains a cross reference of all symbols that have been referenced in an INTERN, EXTERN, or OVRLAY statement so that you can see in which .OBJ files these references occur. GLOBAL is particularly useful when you want to find out what references are made to symbols *between* files.

GLOBAL does not support library files.

11.1 GLOBAL COMMAND LINE

The GLOBAL command line takes this form:

```
GLOBAL{/switches} filespec1,filespec2{,...filespecN}
```

where ***/switches*** are optional and affect the format of the information in the listing file. ***Filespec1...filespecN*** is a list of file specifications that select the .OBJ files for which you want the global cross reference.

If you omit the extension from a file specification, GLOBAL uses the default extension of .OBJ.

GLOBAL produces the listing file in the account and device you are logged into with the name of the first file specification on the command line and a .GLB extension.

11.1.1 Continuation Lines

If there are too many file specifications to fit on one line, you may end the command line with a comma. GLOBAL continues to accept file specifications as long as the last filespec on the line ends with a comma. If the last filespec on the line ends with a comma, GLOBAL prompts you with an asterisk for more filespecs.

11.1.2 GLOBAL Options

You may request the following options by including the appropriate switches on your command line.

Line Width Options (default is 80 characters):

OPTION	FUNCTION
/W	Wide listing (same as /W:130). Produces a listing file that may have up to 130 characters on a line.
/W:n	Specifies ***n*** characters per line.

Page Length Options (default is 60 lines):

OPTION	FUNCTION
/L	Long listing (same as /L:80).
/L:n	Specifies ***n*** lines per page.

Each switch must begin with a slash. For example:

```
GLOBAL/W/L MAIN,SUB1,SUB2 RETURN
```

11.2 SAMPLE GLOBAL DISPLAY

As GLOBAL processes the specified files, it displays a message telling you so ("Processing filespec"). After it processes all files, GLOBAL produces a .GLB file—as it works, it displays the name of the file it is building and displays a dot for each disk block it outputs. For example:

```
Building MAIN.GLB ....
```

This file has the same name as the first file you specified on the GLOBAL command line.

Below is a sample GLOBAL display:

```
GLOBAL MAIN,SUB1,SUB2,SUB3 RETURN

== AM-100/L Global Cross Referencer Version X.X(xxx) ==

Processing MAIN.OBJ
Processing SUB1.OBJ
Processing SUB2.OBJ
Processing SUB3.OBJ

Building MAIN.GLB.....

Global file finished
```

If GLOBAL found any reference errors, it tells you so. For example:

```
Global file finished, 2 errors exist
```

11.3 SAMPLE LISTING DISPLAY

The listing file that GLOBAL produces lists each defined symbol and what .OBJ file the symbol was referenced in. The listing tells you whether the symbol was referenced as an internal symbol (I) via an INTERN pseudo opcode, an external symbol (E) via an EXTERN or AUTOEXTERN pseudo opcode, or an overlay symbol (0) via an OVRLAY pseudo opcode.

Here is a portion of what a GLOBAL listing file might look like:

```
AM-100/L Global Cross-Reference  (Version X.X(xxx))


                                    M S S S
                                    A U U U
                                    I B B B
                                    N 1 2 3
                                    - - - -
ALPHA                               I E . E
BETA                                I . E .
ZETA                                I O . .
```

The listing file above tells us:

1. The symbol ALPHA appeared in an INTERN statement in the file MAIN.OBJ and in EXTERN statements in the files SUB1.OBJ and SUB3.OBJ;

2. The symbol BETA appeared in an INTERN statement in MAIN.OBJ and in an EXTERN statement in SUB2.OBJ;

3. The symbol ZETA appeared in an INTERN statement in MAIN.OBJ and in an OVRLAY statement in SUB1.OBJ.

11.4 GLOBAL ERROR MESSAGES

You may see the following error messages when using GLOBAL:

?Cannot OPEN filespec - not found

GLOBAL could not find the file you specified. Make sure you are logged into the correct account on the right device.

?Fatal error - Incompatible object file version

You are using an obsolete object file. Make sure you have not inadvertently mixed object files from a previous version of the operating system with your current assembler software. Make sure the SYSLIB.LIB file you are using is the correct version for your assembly language system.

?Undefined switch /X - ignored

You specified an invalid switch. The only switches GLOBAL recognizes are the /L and /W switches.

APPENDIX A

SUMMARY OF PROGRAM OPTIONS

The following sections summarize the command line format and options available for each of the assembly language programming system components: M68, LNKLIT, SYMLIT, LIBLIT, and GLOBAL.

If you need to list more files on a command line than will fit, LNKLIT, SYMLIT, LIBLIT, and GLOBAL allow you to continue the list of files on the next line by ending the last filespec with a comma and then pressing RETURN. The program then displays an asterisk and waits for more filespecs—it will continue to accept files as long as the last filespec on the line terminates with a comma.

A.1 M68

The format for the assembler command line is:

```
M68 filespec{/switch}
```

The table below lists the M68 option switches. You do not have to specify the generate a listing.

M68 Option Switches

OPTION	FUNCTION
/B text	Generates a bottom footer line on every page of the listing using the rest of the text on the command line following the /B switch as title information. For example: **`M68 DEVCPY/B Version A00`** RETURN generates a listing file of which every page contains the bottom line title: "Version A00." /B must be the last switch on the command line.

(Continued)

M68 Option Switches (Continued)

OPTION	FUNCTION
/C	Includes conditionals in the listing. Conditionals are normally suppressed.
/E	Writes to the assembly listing only those lines that contain an error.
/L	Generates a list file by calling Phase 3 during the assembly. Creates the output file with the same name as your source file, but a .LST extension. You may modify the name of your listing file by using the OBJNAM pseudo opcode in your source program. See Chapter 5 for information on OBJNAM.
/M20	Allows the assembler to accept 68020 instructions and addresssing modes. (A syntax error S will be generated on assembly if your program contains such instructions or addressing modes, and you do not use the /M20 switch.) For information on the 68020 extensions, see the latest version of the *Alpha Micro Instruction Set Manual*.
/M40	Allows the assembler to accept 68040 instructions and addresssing modes. (A syntax error S will be generated on assembly if your program contains such instructions or addressing modes, and you do not use the /M40 switch.) For information on the 68040 extensions, see the latest version of the *Alpha Micro Instruction Set Manual*.
/N	Does not force the assembler to fold all lower case letters to upper case letters. If this switch is not used, the assembler defaults to folding lower case letters to upper case letters.
/O	Uses the current object file by omitting Phases 1 and 2.
/R	Generates a cross reference, which appears at the end of the assembly listing. See Chapter 8 for information on the cross reference listing.
/S	Keeps EXTERNs from displaying during phase two of assembly.
/T	Displays the assembly listing on your terminal instead of writing it to a disk file.
/V{a}:x	Allows you to specify a value on the M68 command line which can be examined during the assembly process. ***a*** specifies the type of value specified, and ***x*** is the value. See Chapter 8 for more information.
/X	Lists all macro expansions in your assembly listing. M68 expansions are normally suppressed.

A.2 LNKLIT

The format of the LNKLIT command is:

```
LNKLIT {/switches}filespec1{filespec2,..filespecN}{/switches}
```

The first file specified cannot be a library file or an overlay file. You may specify multiple switches by preceding each switch with a /. The option switches LNKLIT understands are:

LNKLIT Option Switches

OPTION	FUNCTION
/B	Don't force word alignment of .OBJ modules.
/D	Generates a Run Time Initializer (.RTI) file. See "The Run Time Initializer File," in this chapter for information on the .RTI file. *Operation switch.*
/E	Includes equated symbols in the symbol table file. You must use /E with the /M or /S switch. *Operation switch.*
/L	Designates a library file. See "Library and Optional Files" in this chapter for information on library files.
/M	Generates a load map (.MAP) file. See "The Load Map File" in this chapter for a discussion of the load map. *Operation switch.*
/N	Suppresses /P switch. *Operation switch.*
/O	Designates an optional file. See "Library and Optional Files" in this chapter for information on optional files.
/P	Generates program (.LIT) and overlay (.OVR) files. The default switch. *Operation switch.*
/R	Designates a required file. The default switch. Cancels the /L and /O switches.
/S	Generates a symbol table (.SYM) file. *Operation switch.*
/U	Don't automatically include SYSLIB.LIB in linkage.
/X	Generates an Extended DSECT, allowing a maximum DSECT size of 64K rather than the 32K default value. Refer to Chapter 5 for information on DSIZE value. *Operation switch.*

A.3 SYMLIT

The format for calling SYMLIT is identical to the LNKLIT command line:

```
SYMLIT {/switches}filespec1{,filespec2,...filespecN}{/switches}
```

The first file specified cannot be a library file or an overlay file. You may specify multiple switches by preceding each with a /. The table below lists the SYMLIT option requests.

SYMLIT Option Switches

OPTION	FUNCTION
/B	Don't force word alignment of .OBJ modules.
/D	Generates a Run Time Initializer (.RTI) file. See "The Run Time Initializer File" in this chapter for information on the .RTI file. *Operation switch.*
/E	Includes equated symbols in the symbol table file. You must also use this switch with /M to tell SYMLIT to include equated symbols in the load map. *Operation switch.*
/L	Designates a library file. See "Libary and Optional Files" in this chapter for information on library files.
/M	Generates a load map (.MAP) file. See "The Load Map File" in this chapter for a discussion of the load map. *Operation switch.*
/N	Suppresses /P switch. *Operation switch.*
/O	Designates an optional file. See "Library and Optional Files" in this chapter for information on optional files.
/P	Generates program (.LIT) and overlay (.OVR) files. The default switch. *Operation switch.*
/R	Designates a required file. The default switch. Cancels the effect of a /L or /O switch.
/S	Generates a symbol table (.SYM) file. *Operation switch.*
/U	Don't automatically include SYSLIB.LIB in linkage.
/X	Allows a maximum DSECT area of 64Kb rather than the default 32Kb value by allowing use of full, 16-bit signed offsets. Initializes DSECT location counter to –32768. Refer to Chapter 5 for information on DSIZE value. *Operation switch.*

A.4 LIBLIT

The LIBLIT command line takes the form:

```
LIBLIT{/L} output=input1{,input2,...inputN}
```

A shorthand version allows you to specify a single file as both the input and output file. This is used to update an existing library. The form is:

```
LIBLIT{/L} inout{,input2,...inputN}
```

This is equivalent to:

```
LIBLIT inout=inout{,input2,...inputN}
```

if you do not use the /L switch—otherwise, it is equivalent to:

```
TRM:=inout{,input2...inputN}
```

The input file specification can take the following forms:

filespec
filespec\item1
filespec\(item1,item2,...itemN)
filespec(item2,item2,...itemN)

The \ symbol designates an exception. The parentheses specify a group of object files.

The only LIBLIT switch is /L, which tells LIBLIT to generate a library listing.

If you specify an output file, LIBLIT creates the listing with that name and extension. If you do not specify an output file, LIBLIT sends the library listing to your terminal display.

A.5 GLOBAL

The GLOBAL command line takes this form:

```
GLOBAL{/switches} filespec1,filespec2{,...filespecN}
```

The table below lists the GLOBAL option switches. You may use more than one option by beginning each with a slash.

Line Width Options (default is 80 characters):

OPTION	FUNCTION
/W	Wide listing (same as /W:130). Produces a listing file that may have up to 130 characters on a line.
/W:n	Specifies ***n*** characters per line.

Page Length Options (default is 60 lines):

OPTION	FUNCTION
/L	Long listing (same as /L:80).
/L:n	Specifies ***n*** lines per page.

APPENDIX B

ASCII CHART

The following pages contain the ASCII codes and the characters to which they are assigned in octal, decimal, and hexadecimal. The first page of the chart contains undisplayable control-characters—the rest of the characters are printable.

SYMBOL	OCTAL	DEC.	HEX.	MEANING
NULL	000	000	00	Null (fill character)
SOH	001	001	01	Start of Heading
STX	002	002	02	Start of Text
ETX	003	003	03	End of Text
ECT	004	004	04	End of Transmission
ENQ	005	005	05	Enquiry
ACK	006	006	06	Acknowledge
BEL	007	007	07	Bell code
BS	010	008	08	Back Space
HT	011	009	09	Horizontal Tab
LF	012	010	0A	Line Feed
VT	013	011	0B	Vertical Tab
FF	014	012	0C	Form Feed
CR	015	013	0D	Carriage Return
SO	016	014	0E	Shift Out
SI	017	015	0F	Shift In
DLE	020	016	10	Data Link Escape
DC1	021	017	11	Device Control 1
DC2	022	018	12	Device Control 2
DC3	023	019	13	Device Control 3
DC4	024	020	14	Device Control 4
NAK	025	021	15	Negative Acknowledge
SYN	026	022	16	Synchronous Idle
ETB	027	023	17	End of Transmission Blocks
CAN	030	024	18	Cancel
EM	031	025	19	End of Medium
SS	032	026	1A	Special Sequence
ESC	033	027	1B	Escape
FS	034	028	1C	File Separator
GS	035	029	1D	Group Separator
RS	036	030	1E	Record Separator
US	037	031	1F	Unit Separator
				°

SYMBOL	OCTAL	DEC.	HEX.	MEANING
SP	040	032	20	Space
!	041	033	21	Exclamation Mark
"	042	034	22	Quotation Mark
#	043	035	23	Number Sign
$	044	036	24	Dollar Sign
%	045	037	25	Percent Sign
&	046	038	26	Ampersand
’	047	039	27	Apostrophe
(	050	040	28	Opening Parenthesis
)	051	041	29	Closing Parenthesis
*	052	042	2A	Asterisk
+	053	043	2B	Plus
,	054	044	2C	Comma
-	055	045	2D	Hyphen or Minus
.	056	046	2E	Period
\	057	047	2F	Slash
0	060	048	30	Zero
1	061	049	31	One
2	062	050	32	Two
3	063	051	33	Three
4	064	052	34	Four
5	065	053	35	Five
6	066	054	36	Six
7	067	055	37	Seven
8	070	056	38	Eight
9	071	057	39	Nine
:	072	058	3A	Colon
;	073	059	3B	Semicolon
<	074	060	3C	Left Angle Bracket
=	075	061	3D	Equal Sign
>	076	062	3E	Right Angle Bracket
?	077	063	3F	Question Mark
@	100	064	40	Commercial At
A	101	065	41	Upper Case Letter
B	102	066	42	Upper Case Letter
C	103	067	43	Upper Case Letter
D	104	068	44	Upper Case Letter
E	105	069	45	Upper Case Letter
F	106	070	46	Upper Case Letter
G	107	071	47	Upper Case Letter
H	110	072	48	Upper Case Letter
I	111	073	49	Upper Case Letter
J	112	074	4A	Upper Case Letter
K	113	075	4B	Upper Case Letter
L	114	076	4C	Upper Case Letter
M	115	077	4D	Upper Case Letter
N	116	078	4E	Upper Case Letter
O	117	079	4F	Upper Case Letter
P	120	080	50	Upper Case Letter
Q	121	081	51	Upper Case Letter
R	122	082	52	Upper Case Letter
S	123	083	53	Upper Case Letter
T	124	084	54	Upper Case Letter

SYMBOL	OCTAL	DEC.	HEX.	MEANING
U	125	085	55	Upper Case Letter
V	126	086	56	Upper Case Letter
W	127	087	57	Upper Case Letter
X	130	088	58	Upper Case Letter
Y	131	089	59	Upper Case Letter
Z	132	090	5A	Upper Case Letter
[	133	091	5B	Left Square Bracket
\	134	092	5C	Back Slash
]	135	093	5D	Right Square Bracket
^	136	094	5E	Circumflex
_	137	095	5F	Underline
‘	140	096	60	Grave Accent
a	141	097	61	Lower Case Letter
b	142	098	62	Lower Case Letter
c	143	099	63	Lower Case Letter
d	144	100	64	Lower Case Letter
e	145	101	65	Lower Case Letter
f	146	102	66	Lower Case Letter
g	147	103	67	Lower Case Letter
h	150	104	68	Lower Case Letter
i	151	105	69	Lower Case Letter
j	152	106	6A	Lower Case Letter
k	153	107	6B	Lower Case Letter
l	154	108	6C	Lower Case Letter
m	155	109	6D	Lower Case Letter
n	156	110	6E	Lower Case Letter
o	157	111	6F	Lower Case Letter
p	160	112	70	Lower Case Letter
q	161	113	71	Lower Case Letter
r	162	114	72	Lower Case Letter
s	163	115	73	Lower Case Letter
t	164	116	74	Lower Case Letter
u	165	117	75	Lower Case Letter
v	166	118	76	Lower Case Letter
w	167	119	77	Lower Case Letter
x	170	120	78	Lower Case Letter
y	171	121	79	Lower Case Letter
z	172	122	7A	Lower Case Letter
{	173	123	7B	Left Brace
|	174	124	7C	Vertical Line
}	175	125	7D	Right Brace
~	176	126	7E	Tilde
DEL	177	127	7F	Delete
				°

APPENDIX C

RAD50 CONVERSION

Throughout the operating system, whenever it is important to conserv space, the monitor often stores and retrieves ASCII characters as RAD50 data. For example, filenames within disk directories are stored in RAD50 form.

ASCII characters take up one byte each—using the RAD50 packing algorithm, you can "pack" three ASCII characters into just two bytes.

The ASCII characters you can pack into RAD50 data are shown below. (The octal codes on the right are used to compute the RAD50 values of the characters. Code 35 is unused.)

CHARACTER	RAD50 CODE
blank	0
A-Z	1-32
a-z	1-32
$	33
.	34
0-9	36-47

The chart below gives you a method for manually converting RAD50 data back into its ASCII character equivalents, and also for converting ASCII characters into RAD50 data. In case you are interested in the actual algorithm used to pack three ASCII characters into two RAD50 bytes, here is a summary:

1. Multiply the first character code by 3100 (octal). (To find out the character's code, consult the short table above.) 3100 is 50 (octal) X 50 (octal)—hence, the name RAD50.

2. Multiply the second character code by 50 (octal) and add it to the first number.

3. Add the third character code to the result from Step #2. This final sum is the RAD50 result.

To convert RAD50 data back into ASCII characters, reverse the sequence above, subtracting instead of adding.

The PACK and UNPACK monitor calls are available for you use to convert ASCII characters into RAD50 data and vice versa. Consult the *AMOS Monitor Calls Manual* for information on all monitor calls.

C.1 USING THE CONVERSION CHART

The following procedures use the conversion chart shown in the next section.

To Convert ASCII Characters Into RAD50

1. Look in the first column on the left for the first ASCII character you want to pack. Write down the octal number associated with it.

2. Look in the second column for the second ASCII character you want to pack. Write down the octal number associated with it beneath the first number you wrote down. Now, add these numbers. Remember, add them in ***octal***.

3. Look in the third column for the third ASCII character you want to pack. Write down the octal number associated with it beneath the sum of the first two numbers. Add the new number.

The final number is the 16-bit RAD50 result.

To Convert RAD50 Data into ASCII Characters

1. Write down the RAD50 octal data you want to convert.

2. Look in the first column on the left for the largest octal number that can be subtracted from your RAD50 data without producing a negative number. The character associated with that number is the first ASCII character. Now, subtract the octal number from the RAD50 data.

3. Look in the second column for the largest octal number that can be subtracted from your new result without producing a negative number. The character associated with that number is the second ASCII character. Subtract the octal number from the current RAD50 result.

4. Look in the third column for the largest octal number that can be subtracted from your new result without producing a negative number. The character associated with that number is the final ASCII character.

Conversion Examples

To convert the ASCII characters "DSK" into RAD50:

1. Look for "D" in the first column. Write down its associated number:

```
014400
```

2. Look for "S" in the second column. Write down its associated number, and add it to the octal data above:

```
 014400
+001370
-------
 015770
```

3. Look for "K" in the third column. Write down its associated number and add it to the result above:

```
 014400
+001370
-------
 015770
+000013
-------
 016003
```

The octal, 16-bit RAD50 representation of the ASCII characters "DSK" is 016003.

To convert the RAD50 data 016003 into three ASCII characters:

1. Look for the largest number in the first column you can subtract from 016003 without producing a negative number—014400. The character associated with 014400 is "D." Subtract the number:

```
 016003
-014400
-------
 001403
```

2. Look for the largest number in the second column you can subtract from 001403 without producing a negative number—001370. The character associated with 001370 is "S." Subtract the number:

```
 016003
-014400
-------
 001403
-001370
-------
 000013
```

3. Look for the result in the third column. The character associated with 000013 is "K."

The RAD50 number 016003 represents the three ASCII characters "DSK."

C.2 CONVERSION CHART FOR PACKING AND UNPACKING RAD50 DATA

CHARACTER #1		CHARACTER #2		CHARACTER #3	
Blank	000000	Blank	000000	Blank	000000
A	003100	A	000050	A	000001
B	006200	B	000120	B	000002
C	011300	C	000170	C	000003
D	014400	D	000240	D	000004
E	017500	E	000310	E	000005
F	022600	F	000360	F	000006
G	025700	G	000430	G	000007
H	031000	H	000500	H	000010
I	034100	I	000550	I	000011
J	037200	J	000620	J	000012
K	042300	K	000670	K	000013
L	045400	L	000740	L	000014
M	050500	M	001010	M	000015
N	053600	N	001060	N	000016
O	056700	O	001130	O	000017
P	062000	P	001200	P	000020
Q	065100	Q	001250	Q	000021
R	070200	R	001320	R	000022
S	073300	S	001370	S	000023
T	076400	T	001440	T	000024
U	101500	U	001510	U	000025
V	104600	V	001560	V	000026
W	107700	W	001630	W	000027
X	113000	X	001700	X	000030
Y	116100	Y	001750	Y	000031
Z	121200	Z	002020	Z	000032
$	124300	$	002070	$	000033
.	127400	.	002140	.	000034
Unused	132500	Unused	002210	Unused	000035
0	135600	0	002260	0	000036
1	140700	1	002330	1	000037
2	144000	2	002400	2	000040
3	147100	3	002450	3	000041
4	152200	4	002520	4	000042
5	155300	5	002570	5	000043
6	160400	6	002640	6	000044
7	163500	7	002710	7	000045
8	166600	8	002760	8	000046
9	171700	9	003030	9	000047

APPENDIX D

SPECIAL ISSUES ON THE MC68040

The MC68040 processor used in the AM-4000 and RoadRunner ’040 contains more internal instruction cache (4096 bytes) than earlier MC68030-based processors. Increased internal cache improves performance; however, if your software does not properly manage this instruction cache, it could be adversely affected.

When the instruction cache is enabled—as it is during normal system operation—any memory location which is executed by the processor is loaded into the instruction cache. This improves performance for those cases where the same location is executed again, since the instruction will be read from cache instead of slower main memory.

D.1 THE PROBLEM AND WHY IT’S A PROBLEM

The problem arises if you write programs that modify instruction opcodes. When the program executes the first time, the opcode is copied into cache. If the opcode is then modified by a data instruction, only the copy in memory is changed. The copy in cache is not affected. This is because data operations, such as MOV, do not update the instruction cache on Motorola 68000-family processors. If the opcode is executed a second time, the CPU will pull it from cache, and execute the unchanged version. Debugging this problem is made more difficult because the instruction cache holding the unmodified opcode may get overwritten by other incoming instructions. If this happens, the program may work in certain cases, but not in others—it will depend on the order of opcode execution. Even doing seemingly innocuous things, such as adding debugging statements, can make the problem disappear and reappear. This programming technique of modifying instructions is known as self-modifying code.

In this context, self modifying code is any code which is written to memory as data, such as when read from disk to memory, and then executed as instructions. AMOS handles the most common cases, such as reading data from disk or during the FETCH monitor call, but cannot handle cases in your software where you are handling program overlays or building instructions on the fly. Such code is often present in "trick" code like that used to perform SSD-based software protection, although it may be present anywhere.

This issue was present on 68030 based systems, but the limited cache size on that processor made it a much smaller problem. Tricks to flush the cache that worked on the 68030, such as forcing a context switch, will probably not work reliably on the 68040.

D.2 WHAT YOU MUST DO

The only sure way to address this issue is to flush the instruction cache after loading instructions into memory, but before executing those instructions. AMOS includes two monitor calls. ICON and ICOFF, which turn the instruction cache off and on. These calls are described in the *AMOS Monitor Calls Manual*.

Remember to turn the instruction cache back on! If you don't, system performance will be seriously impaired.

D.3 ONE MORE CAUTION

The 68040 also has internal registers called Transparent Translation Registers (also present on the 68030) that you must be sure not to modify. This is far less common than instruction cache issues, but will cause system failure if modified.

APPENDIX E

USING AMSORT.SYS

AMSORT.SYS is a sort module included with the AMOS operating system which can be easily called from assembler or C programs. It uses a binary insertion sort method, which yields a stable sorting sequence. This means that two different records will be returned in the same order as they were presented to AMSORT if their sort keys compare equally. AMSORT can transparently handle more data than can fit in memory, automatically paging data in and out of memory as needed without extra work on the programmer's part. The number of keys supported is limited only by the amount of free memory. AMSORT calls programmer-defined routines to read and write data, providing complete flexibility for data sources and sinks.

AMSORT can deal with the following key data types:

- 7-bit ASCII strings. It can optionally support 8-bit characters and the extended collating sequence defined in the job's language definition file. Character by character comparisons are done, proceeding from lower locations to higher ones. If the optional extended collating flag is set, the key is translated to and from the collating sequence characters to perform the comparison.

- AMOS floating point numbers. AMSORT uses the FCMP call to compare the keys.

- Binary data. AMSORT uses integer data in AlphaBASIC unsigned binary data format.

Non-key data in a record can be in any format.

AMSORT is easy to use. The following steps give an overview of the process:

1. Load AMSORT into user memory if it is not already loaded in user or system memory.

2. Use the AMCHK macro to verify that AMSORT.SYS is a current version.

3. Point A5 at an impure area. Details about this impure area are given below.

4. Check the active language for the job, and decide whether or not to call the 8-bit character collation initialization sequence in AMSORT. If so, set the COLL8 flag using the XTDINI macro.

5. Define the area in memory that can be used by AMSORT for sorting purposes. This area is separate from, but can be contiguous with, AMSORT's impure area.

6. Set up the key list in the impure area using the KEYOFF macro.

7. Clear any unwanted features flags in the impure area using the FLGCLR macro.

8. Set the maximum record size of the records being sorted in the impure area.

9. Set the addresses of your own input and output routines in the impure area.

10. Call AMSORT.

The macros mentioned above are defined in the AMSYM.UNV file, which should be SEARCHed in your source file.

E.1 LOCATING AND VERIFYING AMSORT.SYS

AMSORT can be located in memory by using the SRCH and FETCH monitor calls. The AMCHK macro returns with the Z flag set if the AMSORT.SYS module is usable.

E.2 THE IMPURE AREA

AMSORT uses an impure area which must be indexed by A5 for variable storage and the key list table. The variable storage is an area of fixed length that holds variables you set, as well as internal variables private to AMSORT. You are responsible for building the key list table at the end of the variable storage area with the help of the KEYOFF macro. The size of the impure area which must be allocated is calculated by adding the minimum impure area size to N * KEYSIZ, where N is the number of sort keys to be defined, and KEYSIZ is the size of a key table entry, currently 14. bytes. To find the minimum impure area size, locate AMSORT.SYS in memory, and use:

```
KEYOFF    An, Dn
```

where ***An*** is an address register holding the address of the AMSORT.SYS module in memory, and ***Dn*** receives the impure offset value to the key list table. Dn also represents the minimum impure area size. Add N * KEYSIZ to Dn, and point register A5 to an area of that size or larger to define the impure area. The start of the key table list, Am, is then given by:

```
LEA  Am, 0(A5)[Dn]
```

where ***A5*** indexes the impure area. Each key table entry is defined as follows:

SIZE	lword	Length of key in bytes, maximum = 65,535 bytes	
LOCATION	lword	Offset into the record for this key. The first byte of the record is byte zero.	
SPARE	lword	Reserved, set to null	
DIRECTION	byte	Sorting direction.	
		bit 7 set	Ascending
		bit 7 clear	Descending
		All other bits	Reserved, set to zero.
TYPE	byte	Type of key:	
		0	String
		1	AMOS 6-byte floating point
		2	AlphaBASIC binary
KEYSIZ = 14.		Size of a key table entry	

The list is terminated by a key table entry with a SIZE set to zero. The key table list must be set up before AMSORT is called.

Within the impure area are two variables: SM.FRE(A5), which defines the start of the contiguous area which AMSORT can use for sorting; and SM.END(A5), which defines the address of the last word of the area. Both addresses must be even addresses. AMSORT will longword align these two values, so they can change after AMSORT is called. The maximum size of this area is 32KB. If a larger block is defined, AMSORT will shrink the area during initialization by altering the SM.END(A5) value.

E.3 SETTING THE RECORD SIZE

The maximum size of a record is set into the longword at SM.REC(A5). Only the lower word is used, providing a maximum record size of 65,535 bytes. The total of each key's SIZE and LOCATION must be less or equal to SM.REC(A5).

E.4 SETTING FLAGS

The FLGCLR macro is then called to initialize the flags in AMSORT's impure area.

If you do not want AMSORT to use the job's language definition file's extended 8-bit collating sequence, clear the byte at COLL8(A5). If you want the extended collating sequence used, place a non-zero byte in COLL8(A5), and initialize 8-bit processing by calling the XTDINI macro. If XTDINI is not called when it should be, or is called when the LDF does not contain an extended collating sequence, the sorted records returned will be corrupted. XTDINI needs to be called only when the first (or a new) impure area for AMSORT is being defined.

E.5 INPUT AND OUTPUT ROUTINES

You define your input and output routines to AMSORT by placing their addresses in SM.IN(A5) and SM.OUT(A5) respectively. Any registers used in your routines must be preserved by those routines. AMSORT will keep calling your input routine until you return a zero in register D0. You must place exactly SM.REC(A5) bytes of data (padded with nulls if needed) into the area indexed by A1. Return non-zero in register D0 and a zero in D1 until the last record is being processed, then return the last record with registers D0 and D1 set to zero. If your input routine is called again, immediately return a -1 in register D1 to signal an error condition.

After you have sent the last record to AMSORT, AMSORT will start calling your output routine with register A1 pointing to the record you are to receive. Register D2 will be set to SM.REC(A5) to help you do the transfer from AMSORT's buffer to your data sink. After AMSORT has processed all the records, it will call your output routine again with A1 cleared to zero. On return from that call, AMSORT will clean up after itself, and return to the calling program.

E.6 EXAMPLE PROGRAM

```
        SEARCH  SYS
        SEARCH  SYSSYM
        SEARCH  AMSYM

NITEMS  =       9.                      ; number of data items
RECSIZ  =       5                       ; record size
NKEYS   =       1                       ; number of keys

        PHDR    -1,,                    ; This program is not reentrant or reusuable.
                                        ; Initializing COUNT to zero at the start
                                        ; would make it reusable, and making
                                        ; OUTLST be in another impure area
                                        ; would make it reentrant.
        SRCH    AMNAME, A2              ; look for AMSORT.SYS in memory
        BEQ     10$                     ; ....br if found....
        TYPECR  <?AMSORT.SYS not in memory!>   ; ....else error, abort
        EXIT

10$:    AMCHK   A2                      ; version check
        BEQ     20$
        TYPECR  <?AMSORT.SYS is not current, and cannot be used>
        EXIT

20$:    KEYOFF  A2, D6                  ; D6 holds min impure size
        ADD     #KEYSIZ*NKEYS, D6       ; add key table size
        ADD     #4, D6                  ; allow for terminating null
        PUSH    D6
        PUSH
        GETMEM  @SP                     ; get module for impure area
        BEQ     30$
        TYPECR  <?Insufficient memory for sorting!>
        EXIT

30$:    POP     A5                      ; A5 set to impure area
```

```
        POP                             ; stack restored
        GETIMP  1024., A4               ; allocate 1kbyte of sorting space
        MOV     A4, SM.FRE(A5)          ; set in start of sort space into impure area
        ADD     #1024., A4              ; set upper bound of block....
        MOV     A4, SM.FRE(A5)          ; ....into impure area

        LEA     A3, INSBR               ; a3 -> my input routine
        MOV     A3, SM.IN(A5)           ; ....set into impure area
        LEA     A3, OUTSBR              ; a3-> my output routine
        MOV     A3, SM.OUT(A5)          ; ....set into impure area

        MOV     #RECSIZ, SM.REC(A5)     ; test data record is 5 bytes long
                                        ; (includes terminating null in this case)

        FLGCLR  A5                      ; clear flags

        CLRB    COLL8(A5)               ; default to ignore 8-bit collating
        MOV     SYSTEM, D7              ; D7 holds SYSTEM lword
        AND     #SY$LNG, D7             ; is .LDF file supported????
        BEQ     40$                     ; ....no, use default, br
        GTLANG  A3
        CMPW    LD.XTV(A3), #^H05A5A    ; extended .LDF present????
        BNE     40$                     ; ....no, use default, br
        MOVB    #1, COLL8(A5)           ; set up for extended collating
        XTDINI  A2                      ; initialize...only if extended collating

40$:    KEYOFF  A2, D6                  ; d6 holds key table offset address
        LEA     A3, 0(A5)[D6]           ; A3 indexes the area
        MOV     #4, (A3)+               ; key is four bytes long....
        CLR     (A3)+                   ; ....at byte zero in record....
        CLR     (A3)+                   ; reserved, must be zero
        CLRB    (A3)+                   ; ascending order
        CLRB    (A3)+                   ; string type
        CLR     (A3)+                   ; end of list

        CALL    @A2                     ; call AMSORT!

        TYPECR  <The sorted list:>
        MOV     #NITEMS-1, D3           ; d3 holds items less DBF adjustment
        LEA     A3, OUTLST              ; a3 -> list
50$:    TTYL    @A3                     ; output item
        CRLF
        ADD     #RECSIZ, A3             ; a3 -> next item
        DBF     D3, 50$                 ; loop until done
        EXIT

INSBR:  PUSH    A3                      ; save registers I use
        LEA     A3, COUNT               ; a3 -> items processed
        CLR     D0                      ; preclear
        MOVB    @A3, D0                 ; get count
        CMPB    D0, #NITEMS             ; if all processed....
        BLOS    10$
        MOV     #-1, D1                 ; ....error, signal AMSORT, br
        BR      40$
10$:    MOV     D0, D1                  ; save count
        INC     D0                      ; inc count here....
        INCB    @A3                     ; ....and in memory
        CMPB    D0, #NITEMS             ; all done????
        BLO     20$                     ; ....no, br
```

```
        CLR     D0                      ; yes, so signal AMSORT as well
20$:    LEA     A3, SRTLST              ; a3 -> buffer area
        MUL     D1, #5                  ; d1 converted to offset
        ADD     D1, A3                  ; a3 -> area to hold item
30$:    MOVB    (A3)+, (A1)+            ; move from our buffer to AMSORT s
        BNE     30$                     ; ....until done
        CLR     D1                      ; signal "no error"
40$:    POP     A3                      ; restore registers
        RTN                             ; return to AMSORT

OUTSBR:
        SAVE    A3, D0, D1              ; save registers
        LEA     A3, COUNT               ; a3 -> count
        CLR     D0                      ; preclear
        MOVB    @A3, D0                 ; get count of items to be processed
        BEQ     20$                     ; if all done, br
        DECB    @A3                     ; dec count
        MOV     #NITEMS, D1             ; d1 holds max # items
        SUB     D0, D1                  ; invert about that number
        MUL     D1, #5.                 ; convert item number into offset
        LEA     A3, OUTLST              ; a3 -> buffer area
        ADD     D1, A3                  ; a3 -> place to put item
10$:    MOVB    (A1)+, (A3)+            ; move from AMSORT's buffer to ours
        BNE     10$                     ; ....until done
20$:    REST    A3, D0, D1              ; restore used registers
        RTN                             ; return to AMSORT

AMNAME:
        RAD50   /AMSORTSYS/
COUNT:
        BYTE    0

SRTLST:
        ASCIZ  /GGGG/
        ASCIZ  /BBBB/
        ASCIZ  /BBBC/
        ASCIZ  /1234/
        ASCIZ  / SPC
        ASCIZ  /.PER/
        ASCIZ  /AAAA/
        ASCIZ  /aa  /
        ASCIZ  /aaa /

OUTLST:
        BLKB   NITEMS * RECSIZ

        END
```

Document History

Revision A00 - AMOS/L Release 1.0 - (Printed 6/82)

Converted from original book for AMOS/T operating system, DWM-00100-43, and re-issued under new part number, DSS-10007-00 for first AMOS/L release.

Revision A01 - AMOS/L Release 1.1 - (Printed 3/83)

Added the ASMERP and ASCIIZ statements to Chapter 5, and updated chapters 6 and 7 to reflect expanded capabilities of the software.

Revision 00 - AMOS Release 2.0 - (Printed 3/88)

Converted from original book for AMOS/L operating system, DSS-10007-00, and re-issued under a new part number, DSO-00052-00, for AMOS 2.0 release. Added information about Run Time Initializer, 32-character symbol length and /N switch.

Revision 01 - AMOS Release 2.2 - (Printed 4/91)

Added updates to Chapters 4, 5, and 9 to document AMOS 2.2 changes to BYTE and WORD range checking, and the new WORD64 and WORD32 compiler directives.

Revision 02 - AMOS Release 2.3 - (Printed 9/96)

Added Appendices D and E. Small changes to Chapters 4 and 8 and Appendix A.

INDEX

.GLB file 2-3, 11-1
.IPF file 2-3, 8-1
.LIB file 2-3, 5-14, 9-1, 10-1
.LIT file 2-2, 5-13, 8-1, 9-1
.LST file 2-2, 8-2, 10-3
.M68 file . . . . . 2-1 to 2-2, 8-1
.MAP file 2-3, 9-8
.OBJ file 2-1, 5-13, 8-1, 9-1, 11-1
Multiple 5-13, 10-1
.OVR file 2-2, 5-13, 5-16, 8-1, 8-12, 9-1
.RTI file . . . . . 2-4, 9-3, 9-6, 9-9, A-3 to A-4
.SYM file . . . . . 2-3, 9-3 to 9-5, A-3
.TMP file 2-4
.UNV file 2-4, 5-3, 8-2

68020 extensions 1-2, 8-3, 8-11, A-2
68040 extensions 8-3, A-2

Absolute code 4-7, 5-6
ASECT 4-5, 5-6
AlphaFIX . . . . . 1-3, 2-2 to 2-3, 9-4
Argument 6-3
Concatenation 6-4
ASCII 4-3, 5-11, B-1
Character set 5-11
ASCIZ 5-11
ASECT 4-7, 5-6
ASMERP 5-9
ASMMSG 5-8
Assembled program 2-1
Assembly control statement 3-4, 5-1
Assembly listing 2-2, 8-6
Format 8-7
Assembly location counter 4-3, 4-6, 5-6
Assembly sections 4-7, 5-6
AUTOEXTERN 5-14, 9-1

Binary file 2-2, 9-1
Blank lines 3-5

BLKB 5-12
BLKL 5-12
BLKW 5-12
BYTE 5-9

Comments 3-2, 3-5, 6-4
Blank lines 3-5
Field 3-2
Leading symbol 3-2
Condition code 7-1
Conditional assembly 3-4, 7-1
Block format 7-1
Condition code 7-1
Defining 7-1
Directives 7-1
ENDC 7-1
Examples 7-3
Format 7-1
IF 7-1
IFF 7-3
IFT 7-3
IFTF 7-3
Nesting 7-3
Single-line format 7-1
Subconditionals 7-3
Continuation lines 9-2, 9-5
COPY 5-1, 5-13
File 3-5, 5-13
Search pattern 5-2
CREF 5-7, 8-8

Data generation 3-3
Default statement 3-3
Pseudo opcodes 5-9
Statement 3-3
Debugger . . . . . 1-3, 2-2 to 2-3
Default number base 5-7
DEFINE 6-2
DSECT . . . . . 4-7 to 4-8, 5-6, 9-9, A-4
.RTI file 4-8
Generating .RTI file 4-8
GETRTI monitor call 4-8
Initialization 4-8
DSIZE 5-6
Dummy arguments 6-3

END 5-8
ENDC 7-1
ENDM 6-2
ENDMX 6-6

Equate statement 3-3
Equated symbols 9-6, A-4
Error codes 8-10
Error messages
GLOBAL 11-4
LIBLIT 10-4
LNKLIT 9-10
M68 8-10
SYMLIT 9-10
EVEN 5-7
Expression 4-3, 4-5
Absolute 4-5
Defined 4-3
Evaluation 4-3, 6-5
External 4-5
Relocatable 4-5
EXTERN . . . . . 5-12 to 5-13, 5-15, 9-1, 11-1
External symbols 5-13

FETCH monitor call 5-13
Files
Assembly listing 2-2, 8-6
Binary 2-2, 9-1
COPY 5-13
Global cross reference . . . . . 2-2 to 2-3, 11-3
Intermediate 2-1
Interphase work file 2-3
Library 2-3, 5-14, 9-1, 9-7, 10-1
Library listing 10-3, A-5
Listing 2-2, 5-5, 8-6
Load Map 2-3, 9-8
Multiple .OBJ files 5-13, 10-1
Names 5-4
Object 2-1, 9-1, 10-1, 11-1
Optional 9-2, 9-7
Overlay 2-2, 5-13, 5-16, 9-1, 9-6, A-4
Program 2-2, 8-1, 9-1
Required . . . . . 9-3, 9-6, A-3 to A-4
Resolved 2-2, 9-1
Run time initializer . . . . . 2-4, 9-3, 9-6, 9-9, A-3 to A-4
Source . . . . . 2-1 to 2-2, 8-1
Symbol 2-3, 9-4
Temporary . . . . . 2-3 to 2-4
Universal 2-4, 3-5, 5-3

GETRTI monitor call 2-4, 4-8, 9-9
GLOBAL 2-3, 11-1
Command line 11-1, A-5
Continuation lines 11-2
Cross reference 2-3, 11-3

Error messages 11-4
Generate long listing 11-2, A-6
Generate wide listing 11-2, A-6
Listing format 11-2, A-5
Operation 11-2
Options 11-2, A-5
Sample display 11-3

IF 7-1
IFF 7-3
IFT 7-3
IFTF 7-3
Intermediate file 2-1
INTERN . . . . . 5-12 to 5-14, 9-1, 10-3, 11-1
Internal symbols 5-13
Interphase work file 2-3

JMP instruction 4-6

Label 3-2, 6-3
Field 3-2
Multiple 3-2
LIBLIT 1-1, 2-3, 5-14
Command line 10-1, A-5
Continuation lines 10-2
Error messages 10-4
Exceptions 10-2, A-5
Inclusions 10-2, A-5
Library file 9-8
Library listing 10-3
Operation 10-3
Option (/L) 10-3, A-5
Updating a library 10-4
Library file . . . . . 2-3, 5-14, 9-1, 9-7 to 9-8, 10-1
Defined 2-3, 10-1
LIST 5-5, 8-7
Listing file 2-2, 5-5, 8-3, 8-6, A-2
Examining 2-2
Page breaks 5-5
LNKLIT . . . . . 1-1, 2-2 to 2-3, 5-12 to 5-16, 8-6, 9-1
Command line 9-2, A-3
Continuation lines 9-2
Error messages 9-10
Excluding SYSLIB.LIB 9-3, A-3
Extended DSECT 9-3, A-3
Generate .RTI file 9-3, A-3
Generate load map file 9-3, A-3
Generate symbol table 9-3, A-3
Generating program file 9-3, A-3
Including equated symbols 9-3, A-3

Library file 9-3, A-3
Load map file 9-3, A-3
Optional files 9-3, A-3
Options 9-2
Required file 9-3, A-3
Sample 9-4
Suppress program generation . . 9-3, A-3
Symbol table 9-3, A-3
Load map file 2-3, 9-6, 9-8, A-4
Local symbols 4-11, 6-3
Location counter 4-3, 4-6, 5-6
LWORD 5-10

M68 . . . . . 1-1, 2-2, 8-1
68020 extensions 8-3, A-2
68040 extensions 8-3, A-2
Cross reference file 8-6, 8-8
Error codes 8-10
Error messages 8-10
Error reporting 8-3, A-2
Features 1-1
Generate cross reference 8-3, A-2
Generate listing file 8-3, A-2
Generate listing title 8-3, A-1
Include conditionals in listing . . . 8-3, A-2
Listing control 8-7
Listing file 2-2, 8-6
Listing terminal display 8-4, A-2
Macro expansion 8-4, A-2
Operation 8-4
Options 5-7, 8-3, A-1
Output file 9-1
Parameterized assembly . . . . . 8-4 to 8-5, A-2
Phases 8-1
Sample assembly 8-6
Source file 8-1
Suppress .OBJ file 8-3, A-2
Suppress character folding 8-3, A-2
Suppress EXTERN display 8-4, A-2
Machine instructions 3-2
Macros 3-5
Argument concatentation 6-4
Call arguments 6-1
Calls 3-5, 6-1
Code generation 6-1
Comments 6-4
DEFINE 6-2
Defined 3-5, 6-1
Definition 6-2
Dummy arguments 6-3

Ending expansion 6-6
ENDM 6-2
ENDMX 6-6
Examples 6-4
Expression evaluation 6-5
Format 6-2
How to define 6-1
Labels 6-3
Local symbols 6-3
Multiple-line format 6-2
NCHR 6-6
Nested calls 6-11
NEVAL 6-7
NSIZE 6-7
NTYPE 6-6
Passing arguments 6-11
Real arguments 6-1
Sample calls 6-11
Samples 6-8
Single-line format 6-2
Special operators 6-4
Map file 2-3, 9-8
MAYCREF 5-7, 8-8
Monitor calls 1-1
FETCH 5-13, 5-16
GETRTI 9-9

NCHR 6-6
NEVAL 6-7
NOCREF 5-7, 8-8
NOLIST 5-5, 8-7
NOSYM 5-6
NSIZE 6-7
NTYPE 6-6
Number base 4-2
Numbers 4-5
NVALU 5-7

Object file 2-1, 2-3, 9-1
Object file library 5-14, 10-1
OBJNAM 5-4
OFFSET 5-11
Opcode field 3-2
Termination 3-2
Operands 3-2
Field 3-2
Operation switch 9-2
Operators 4-4, 6-4
Summary 4-4
Optional file . . . . . 9-2, 9-6 to 9-8, A-4

Output file 5-4, 9-1, 9-4
Overlay file . . . . . 2-2, 5-12 to 5-13, 9-1
Overlays 5-16, 9-1
OVRLAY . . . . . 5-12 to 5-13, 5-16, 11-1

PAGE 5-5, 8-7
Parameterized assembly 8-5
Program counter 4-6
Program file 3-1, 9-1
PSECT 4-7, 5-6
Pseudo opcodes 3-4, 5-1
ASCII 5-11
ASCIZ 5-11
ASECT 4-7, 5-6
ASMERP 5-9
ASMMSG 5-8
AUTOEXTERN 5-14, 9-1, 11-1
BLKB 5-12
BLKL 5-12
BLKW 5-12
BYTE 5-9
COPY 3-5, 5-1, 5-13
CREF 5-7, 8-8
Data generation 5-9
DEFINE 6-2
Defined 5-1
DSECT 4-7, 5-6
DSIZE 5-6
END 5-8
ENDC 7-1
ENDM 6-2
ENDMX 6-6
EVEN 5-7
EXTERN . . . . . 5-12 to 5-13, 5-15, 9-1, 11-1
IF 7-1
IFF 7-3
IFT 7-3
IFTF 7-3
INTERN . . . . . 5-12 to 5-14, 9-1, 11-1
LIST 5-5, 8-7
LWORD 5-10
MAYCREF 5-7, 8-8
NOCREF 5-7, 8-8
NOLIST 5-5, 8-7
NOSYM 5-6
NVALU 5-7
OBJNAM 5-4
OFFSET 5-11
OVRLAY . . . . . 5-12 to 5-13, 5-16, 11-1
PAGE 5-5, 8-7

PSECT 4-7, 5-6
RAD50 5-11
RADIX 5-7
SEARCH 5-3
Segmentation 5-12
SYM 5-6
UNIVERSAL 3-5, 5-12, 5-15
WORD 5-10
WORD32 5-10
WORD64 5-10

RAD50 . . . . . 1-3, 4-3, 5-11, C-1
Character set 5-11
Conversion C-1
RADIX 5-7
Re-entrant 1-1
Register 6-6
Registers
Symbols 4-6
Relocatable code 1-1, 4-7, 5-6
PSECT 5-6
Required file . . . . . 9-3, 9-6, A-3 to A-4
Run time initializer 4-8
.RTI file 2-4, 4-8

SEARCH 5-3
Segmentation
AUTOEXTERN 5-14
COPY file 5-13
EXTERN 5-15
INTERN 5-14
Library files 5-14
Multiple .OBJ files 5-13
Opcodes 5-12
Overlay files 5-13
OVRLAY 5-16
UNIVERSAL 5-15
Source file . . . . . 2-1 to 2-2, 3-1, 8-1
Source statements 8-1
Statements 3-1
Assembly control 3-4
Comments 3-5
Conditionally assembling 3-4
Data generation 3-3
Delimiters 3-1
Equate 3-3
Machine instructions 3-2
Macro calls 3-5
Macros 3-5
Type 3-1

Subconditionals 7-3
Examples 7-3
IFF 7-3
IFT 7-3
IFTF 7-3
Nesting 7-3
Rules 7-3
Switch 8-2
SYM 5-6
Symbol file 2-3, 9-6, A-4
Symbolic equate 3-3
SYMLIT 2-3, 8-6, 9-4
Command line 9-5, A-4
Continuation lines 9-5
Equated symbols 9-6, A-4
Error messages 9-10
Excluding SYSLIB.LIB 9-6, A-4
Extended DSECT 9-6, A-4
Generate .RTI file 9-6, A-4
Generate load map file 9-6, A-4
Generate program file 9-6, A-4
Generate symbol table file 9-6, A-4
Include equated symbols 9-6, A-4
Library file 9-6, A-4
Optional file 9-6, A-4
Options 9-5, A-4
Required file 9-6, A-4
Run time initializer file 9-6, A-4
Sample 9-7
Suppress program generation . . 9-6, A-4
Symbol file 9-6, A-4
SYS.M68 . . . . . 1-1, 3-5, 5-1
System Program Library account . . 2-2, 9-2

Temporary file . . . . . 2-3 to 2-4
Terms 4-3

UNIVERSAL 5-12, 5-15
Universal file 2-4, 3-5, 5-3
Updating library 10-4
User symbol 3-1
Defined 3-1

WORD 5-10
WORD32 5-10
WORD64 5-10